

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403

anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450

phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông

Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhẫn

Chữ ký của GVHD

KHOA: (Điền tên khoa)

Hà Nội, 01/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng <...> năm <...>

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	4
DANH SÁCH HÌNH VẼ	6
DANH SÁCH BẢNG	7
PHÂN CHIA CÔNG VIỆC	8
1 TỔNG QUAN ĐỀ TÀI	9
1.1 Bối cảnh và lý do chọn đề tài	9
1.2 Mục tiêu của đề án	10
1.2.1 Mục tiêu tổng quát	10
1.2.2 Mục tiêu cụ thể	10
1.3 Phạm vi và giới hạn	11
1.3.1 Phạm vi nghiên cứu và triển khai	11
1.3.2 Giới hạn của đề tài	11
1.4 Phương pháp thực hiện	11
1.5 Đóng góp chính của đề án	12
1.6 Cấu trúc báo cáo	12
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN	14
2.1 Tổng quan IoT và Smart Building	14
2.2 Giao thức truyền thông Zigbee	14
2.2.1 Tổng quan về Zigbee	14
2.2.2 Vai trò thiết bị trong mạng Zigbee	15
2.2.3 Topology mạng vô tuyến	15
2.2.4 Kiến trúc Protocol Stack	16
2.2.5 Địa chỉ và bảo mật trong mạng	17
2.2.6 Quá trình hình thành và gia nhập mạng	17
2.3 Thư viện cluster Zigbee Cluster Library (ZCL)	18
2.3.1 Cấu trúc mô hình ZCL	18
2.3.2 Cơ chế Binding (Liên kết)	19
2.4 Vai trò của Gateway trong hệ thống IoT	19
2.5 Giao thức truyền thông MQTT	19
2.5.1 Kiến trúc Publish/Subscribe	19
2.5.2 Quality of Service (QoS) trong MQTT	20
2.5.3 Retained Messages và Last Will and Testament (LWT)	20
2.6 Giao tiếp nối tiếp UART	20
2.7 Phần cứng và Kit phát triển sử dụng	21
2.7.1 Wireless Starter Kit BRD4001A và BRD4162A	21
2.7.2 Cấu hình phần cứng theo vai trò ứng dụng	22

3	PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	23
3.1	Phân tích yêu cầu hệ thống	23
3.1.1	Yêu cầu chức năng	23
3.1.2	Yêu cầu phi chức năng	23
3.2	Kiến trúc tổng thể hệ thống	24
3.3	Thiết kế mạng và Vai trò thiết bị	25
3.4	Luồng dữ liệu chính trong hệ thống	25
3.4.1	Luồng báo cáo trạng thái (Uplink Flow)	25
3.4.2	Luồng gửi lệnh điều khiển (Downlink Flow)	25
3.5	Thiết kế dữ liệu và Hợp đồng giao tiếp (MQTT Contract)	26
3.5.1	Phân cấp cấu trúc Topics	26
3.5.2	Cấu trúc Envelope bản tin chung	26
3.6	Thiết kế cơ sở dữ liệu và Cloud Backend	27
3.6.1	Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)	27
3.6.2	Thiết kế REST API Endpoints	27
3.7	Các quyết định thiết kế và Trade-offs quan trọng	28
4	TRIỂN KHAI HỆ THỐNG	29
4.1	Triển khai Firmware cho thiết bị Zigbee	29
4.1.1	Triển khai Coordinator / NCP Board	29
4.1.2	Triển khai Router đèn (Z3Light Device)	29
4.1.3	Triển khai Router công tắc (Z3Switch Device)	30
4.1.4	Triển khai cảm biến hiện diện (Occupancy Sensor)	30
4.2	Triển khai Gateway Service nhúng	30
4.2.1	Kiến trúc vận hành C-MQTT trực tiếp	30
4.2.2	Cơ chế ánh xạ Correlation ID	30
4.2.3	Triển khai Rules Engine cục bộ	31
4.3	Triển khai Cloud Backend và quy trình Deployment	31
4.3.1	Cấu hình biến môi trường	31
4.3.2	Triển khai đồng bộ trạng thái và quét Timeout	31
4.3.3	Quy trình Deploy thực tế trên AWS EC2	31
4.4	Luồng vận hành thực tế End-to-End Command	32
4.5	Hướng thiết kế triển khai cập nhật OTA	32
5	KIỂM THỬ VÀ ĐÁNH GIÁ	34
5.1	Kế hoạch kiểm thử	34
5.2	Môi trường và cấu hình kiểm thử	34
5.2.1	Cấu hình phần cứng thử nghiệm	34
5.3	Kết quả thực thi các kịch bản kiểm thử	35
5.3.1	Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị	35
5.3.2	Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển	36
5.3.3	Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy	37
5.4	Đánh giá kết quả kiểm thử	38
5.4.1	Mức độ hoàn thành các tiêu chí hệ thống	38
5.4.2	Phân tích độ trễ của hệ thống	39
5.5	Hạn chế của hệ thống phát hiện qua kiểm thử	39

6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	41
6.1	Kết luận chung	41
6.2	Kiến thức và kỹ năng đạt được	41
6.3	Hướng phát triển tiếp theo	42
6.3.1	Hoàn thiện phần cứng và mạng thiết bị	42
6.3.2	Nâng cấp tính năng cho Gateway nhúng	42
6.3.3	Nâng cấp nền tảng Cloud và Ứng dụng di động	42
6.3.4	Tăng cường An ninh bảo mật	43
	TÀI LIỆU THAM KHẢO	44

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-The-Air (firmware update)
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device
RPi	Raspberry Pi

Viết tắt	Ý nghĩa
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1	Quy trình nghiên cứu và triển khai hệ thống	11
2.1	Star topology – Thiết bị kết nối qua Coordinator trung tâm	15
2.2	Mesh topology – Định tuyến đa chặng linh hoạt	16
2.3	Tree topology – Cấu trúc phân cấp hình cây	16
2.4	Kiến trúc phân tầng Zigbee Protocol Stack	17
2.5	Mô hình tương tác Client/Server trong ZCL	19
2.6	Kiến trúc truyền thông Publish/Subscribe qua Broker trung gian	20
2.7	Kết nối UART vật lý giữa Host và NCP	21
2.8	Hình ảnh bộ kit phát triển EFR32MG12 Wireless Starter Kit	21
3.1	Sơ đồ kiến trúc tổng quan hệ thống Smart Building	24
3.2	Sơ đồ luồng dữ liệu Uplink đồng bộ trạng thái	25
3.3	Sơ đồ luồng điều khiển Downlink từ Cloud xuống thiết bị	26

DANH SÁCH BẢNG

2	Bảng phân chia công việc	8
1.1	Đóng góp chính và ý nghĩa kỹ thuật của đề án	12
3.1	Các yêu cầu chức năng chính của hệ thống	23
3.2	Các module logic chính trong ứng dụng Z3Gateway C	24
3.3	Bảng ánh xạ device type và các cluster ZCL cấu hình	25
3.4	Thiết kế phân lớp các kênh truyền MQTT	26
3.5	Các bảng cơ sở dữ liệu chính của Cloud Backend	27
3.6	Đặc tả các REST API endpoints chính của hệ thống	28
4.1	Các biến môi trường cấu hình chính của Cloud Backend	31
5.1	Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm	35
5.2	Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)	36
5.3	Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)	36
5.4	Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)	37
5.5	Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống	39

PHÂN CHIA CÔNG VIỆC

STT	Công việc	Sinh viên thực hiện
Firmware – Zigbee Devices		
1	<Firmware Coordinator: tạo mạng, Trust Center, ZCL, UART bridge>	<SV>
2	<Firmware Light Device: On/Off Server, LED indicator, state report>	<SV>
3	<Firmware Switch Device: On/Off Client, binding, button control>	<SV>
4	<Firmware Occupancy Sensor: occupancy detection, ZCL report>	<SV>
Gateway Service (Ubuntu / Raspberry Pi)		
5	<Gateway UART module: kết nối COM, đọc line, reconnect>	<SV>
6	<Gateway MQTT bridge: subscribe cmd, publish state/telemetry/ack>	<SV>
7	<Gateway rules engine, ACK routing, retry/timeout>	<SV>
8	<Gateway Admin API (/health, /status, /config, /rules)>	<SV>
Cloud – Firebase		
9	<Firebase setup: Realtime DB / Firestore schema, Cloud Functions>	<SV>
10	<MQTT – Firebase bridge: device state sync, command relay>	<SV>
11	<Event history, logging, device management trên cloud>	<SV>
Mobile App – Flutter		
12	<Flutter app: device list, light on/off control UI>	<SV>
13	<Flutter app: occupancy state display, realtime update>	<SV>
14	<Flutter app: event history, notification>	<SV>
Tổng hợp & Tài liệu		
15	<Tổng hợp và hoàn thiện báo cáo>	<SV 1, SV 2>

Bảng 2: Bảng phân chia công việc

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài xây dựng hệ thống IoT Zigbee Smart Building. Tiếp theo, chương xác định rõ mục tiêu tổng quát, mục tiêu cụ thể, phạm vi nghiên cứu và các giới hạn thực tế của đề án. Cuối cùng, phương pháp thực hiện, những đóng góp chính về mặt kỹ thuật và cấu trúc tổng thể của báo cáo cũng được khái quát hóa nhằm cung cấp cái nhìn toàn cảnh trước khi đi vào các nội dung kỹ thuật chi tiết.

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ mạng không dây và Internet vạn vật (Internet of Things - IoT) đã thúc đẩy việc ứng dụng tự động hóa vào quản lý vận hành tòa nhà. Smart Building (tòa nhà thông minh) không còn là khái niệm lý thuyết mà đã trở thành xu hướng tất yếu nhằm tối ưu hóa lượng điện năng tiêu thụ, nâng cao độ an toàn và mang lại trải nghiệm tiện nghi cho người sử dụng. Trong các hệ thống này, nhu cầu giám sát trạng thái môi trường, phát hiện sự hiện diện của con người và điều khiển các thiết bị phụ tải (như đèn chiếu sáng, công tắc) là những bài toán cốt lõi cần giải quyết.

Đối với mạng kết nối nội bộ (local network) trong tòa nhà, việc sử dụng các công nghệ truyền thông không dây phổ biến như Wi-Fi thường bộc lộ những hạn chế lớn về mặt tiêu thụ năng lượng và khả năng chịu tải của điểm truy cập trung tâm khi số lượng thiết bị lên tới hàng trăm node. Do đó, các công thức truyền thông tiết kiệm năng lượng, hỗ trợ topo mạng lưới (mesh network) như Zigbee được xem là giải pháp tối ưu. Zigbee hoạt động trên tiêu chuẩn IEEE 802.15.4, cho phép các thiết bị tự động định tuyến gói tin qua nhau, giúp mở rộng phạm vi phủ sóng mà không yêu cầu công suất phát lớn, đồng thời giảm thiểu đáng kể chi phí duy trì nguồn pin cho các cảm biến.

Tuy nhiên, các mạng Zigbee cục bộ không thể tự kết nối trực tiếp với các ứng dụng di động hoặc hệ thống quản lý từ xa qua mạng IP diện rộng. Điều này đòi hỏi sự xuất hiện của một thiết bị trung gian gọi là Gateway. Thiết bị Gateway đóng vai trò làm cầu nối chuyển đổi giữa giao thức Zigbee RF tầm ngắn và mạng IP (chạy các giao thức như MQTT, HTTP) kết nối lên Cloud. Sự phối hợp giữa Gateway và hệ thống đám mây (Cloud Backend) cho phép lưu trữ lịch sử vận hành, xử lý các kịch bản tự động hóa phức tạp và cung cấp các giao diện lập trình ứng dụng (API) cho ứng dụng di động của người dùng cuối.

Hơn nữa, các giải pháp smart building thương mại hiện nay thường là các hệ sinh thái đóng (closed ecosystem) của các nhà sản xuất lớn, gây khó khăn cho việc tích hợp thiết bị của bên thứ ba và hạn chế khả năng tùy biến logic điều khiển. Xuất phát từ nhu cầu nghiên cứu một nền tảng quản lý thiết bị mở, có khả năng làm chủ từ firmware thiết bị, logic xử lý tại gateway cho đến kiến trúc cloud backend, đề tài "**Hệ thống IoT Zigbee Smart Building**" đã được lựa chọn để thực hiện. Đề án tập trung nghiên cứu thiết kế và tích hợp đồng bộ các lớp thiết bị, gateway và dịch vụ đám mây nhằm đảm bảo luồng dữ liệu hai chiều nhanh chóng, tin cậy và có khả năng mở rộng.

1.2 Mục tiêu của đề án

Để giải quyết các bài toán đặt ra trong bối cảnh trên, đề án xác định rõ mục tiêu tổng quát và các mục tiêu cụ thể cần đạt được.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là thiết kế, chế tạo và tích hợp hoàn chỉnh một hệ thống IoT quản lý Smart Building dựa trên chuẩn truyền thông không dây Zigbee, kết nối qua Gateway C tới hệ thống Cloud Backend, hỗ trợ giám sát trạng thái thiết bị thực tế gần thời gian thực và tự động hóa điều khiển phụ tải.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát, đề án đề ra các mục tiêu cụ thể cho từng thành phần trong hệ thống: **Phía thiết bị cuối (End Devices):** Thiết kế và nạp firmware cho các thiết bị Zigbee 3.0 dựa trên vi điều khiển EFR32MG12 bao gồm thiết bị đèn (Z3Light), công tắc (Z3Switch) và cảm biến hiện diện (Occupancy/Motion Sensor) hoạt động ổn định trong mạng mesh Zigbee.

Phía Gateway trung gian: Triển khai ứng dụng Z3Gateway dạng tiến trình đơn (single-process) viết bằng ngôn ngữ C chạy trên Linux host. Tích hợp trực tiếp thư viện kết nối MQTT để giao tiếp trực tiếp với Cloud, loại bỏ hoàn toàn các lớp trung gian trung chuyển IPC phức tạp nhằm giảm độ trễ và tăng tính ổn định.

Phía dịch vụ đám mây (Cloud Backend): Xây dựng Cloud Backend sử dụng framework FastAPI (Python 3.12) kết nối cơ sở dữ liệu PostgreSQL. Hệ thống đám mây phải tiếp nhận các bản tin telemetry báo cáo trạng thái, quản lý vòng đời lệnh điều khiển, lưu trữ lịch sử sự kiện và cung cấp hệ thống REST API chuẩn hóa.

Phía ứng dụng giám sát (Dashboard/App): Thiết kế các giao diện mô phỏng dưới dạng API hoặc màn hình quản lý cho phép theo dõi trực quan danh sách thiết bị, trạng thái vận hành hiện tại và gửi lệnh bật/tắt thiết bị đèn.

Về mặt kiểm thử và đánh giá: Xây dựng bộ kịch bản kiểm thử (test cases) end-to-end hoàn chỉnh nhằm đo đặc độ trễ phản hồi của lệnh điều khiển, kiểm chứng tính chính xác của cơ chế đồng bộ trạng thái và ghi nhận bằng chứng vận hành (logs, traces).

1.3 Phạm vi và giới hạn

Việc xác định phạm vi nghiên cứu và giới hạn thực thi giúp đồ án tập trung vào các vấn đề kỹ thuật trọng tâm, tránh phân tán nguồn lực vào các tính năng thương mại phức tạp.

1.3.1 Phạm vi nghiên cứu và triển khai

Đồ án tập trung thực hiện các nội dung sau: **Thiết lập mạng Zigbee cục bộ:** Sử dụng một Coordinator chạy firmware NCP (Network Co-Processor) kết nối qua UART/ASH với Gateway host.

Triển khai kịch bản gia nhập mạng: Thực hiện permit-join, đồng bộ trạng thái thực tế lên cloud (reported state) và truyền đạt lệnh trạng thái mong muốn (desired state) qua giao thức MQTT.

Đồng bộ trạng thái thiết bị và kịch bản tự động hóa cục bộ: Hỗ trợ local automation cơ bản thông qua cơ chế APS Binding trực tiếp giữa công tắc và đèn trong mạng Zigbee mesh.

Theo dõi vòng đời của lệnh điều khiển: Theo dõi chặt chẽ quá trình xử lý lệnh từ Cloud xuống thiết bị qua các pha trạng thái rõ ràng (accepted → queued → sent → executed/failed/-timeout).

Triển khai hệ thống Cloud Backend: Deploy hệ thống trên hạ tầng điện toán đám mây dưới dạng các container Docker được đóng gói và cấu hình hoàn chỉnh.

1.3.2 Giới hạn của đề tài

Do hạn chế về mặt thời gian và trang thiết bị phần cứng, đồ án tồn tại một số giới hạn như sau: **Quy mô thử nghiệm:** Do hạn chế về mặt thời gian và trang thiết bị phần cứng, hệ thống chỉ dừng lại ở quy mô thử nghiệm phòng lab (lab-scale) với số lượng thiết bị demo hạn chế (khoảng 3-5 node), chưa được triển khai thực tế trên quy mô tòa nhà nhiều tầng với mật độ cản trở vật lý cao.

Ứng dụng di động: Ứng dụng di động Flutter và cơ chế điều khiển tích hợp qua giao thức Bluetooth Low Energy (BLE) chỉ dừng lại ở mức thiết kế giao diện tĩnh (placeholders) hoặc mô phỏng logic client, chưa được tích hợp kết nối API thực tế chạy trên thiết bị di động thật trong bản v1 này.

Tính năng nâng cao: Các tính năng nâng cao như cơ chế cập nhật firmware từ xa (OTA Update), bảo mật mã hóa đường truyền MQTT qua TLS (port 8883) và hệ thống xác thực người dùng (Authentication API) chỉ được thực hiện ở mức phân tích thiết kế kịch bản lý thuyết (contracts), chưa có mã nguồn triển khai chạy thực tế.

1.4 Phương pháp thực hiện

Để thực hiện đồ án một cách khoa học, quy trình nghiên cứu và phát triển được chia làm 5 bước tuần tự như được mô tả trong Hình 1.1.

Hình 1.1: Quy trình nghiên cứu và triển khai hệ thống

Quy trình thực hiện bao gồm các giai đoạn cụ thể sau: **Giai đoạn 1 – Khảo sát và nghiên cứu công nghệ:** Nghiên cứu tài liệu kỹ thuật về chuẩn Zigbee 3.0, thư viện ZCL (Zigbee Cluster Library) của Silicon Labs, giao thức truyền thông MQTT và kiến trúc API hướng tài nguyên sử dụng REST.

Giai đoạn 2 – Thiết kế hệ thống: Xác định sơ đồ khối kiến trúc hệ thống 4 lớp, thiết kế cấu trúc cơ sở dữ liệu PostgreSQL, xây dựng hợp đồng trao đổi bản tin MQTT (MQTT Contract JSON Schema) và cấu hình các topic phân cấp.

Giai đoạn 3 – Triển khai chi tiết từng module: Lập trình firmware cho các SoC EFR32MG12; viết mã nguồn Z3Gateway bằng ngôn ngữ C; xây dựng Cloud API bằng FastAPI.

Giai đoạn 4 – Tích hợp hệ thống: Kết nối Gateway host với chip NCP qua UART, kết nối Gateway lên Mosquitto Broker chạy trên container Docker, thiết lập liên kết đồng bộ giữa cơ sở dữ liệu và MQTT Client chạy nền trên Cloud.

Giai đoạn 5 – Kiểm thử và đánh giá: Chạy các kịch bản kiểm thử tích hợp tự động bằng pytest cho backend và thực thi kiểm thử thủ công end-to-end trực tiếp trên phần cứng thật để thu thập log vận hành.

1.5 Đóng góp chính của đề án

Kết quả thực hiện đề án mang lại một số đóng góp chính về mặt giải pháp kỹ thuật tự phát triển thay vì sử dụng các framework đóng gói sẵn của hãng. Bảng 1.1 tóm tắt các đóng góp và ý nghĩa thực tế tương ứng của chúng.

Bảng 1.1: Đóng góp chính và ý nghĩa kỹ thuật của đề án

Giải pháp phát triển	Ý nghĩa thực tế và kỹ thuật
Kiến trúc Gateway C đơn tiến trình tích hợp trực tiếp MQTT client	Loại bỏ hoàn toàn tiến trình trung gian Python IPC cũ, giúp giảm thiểu độ trễ truyền dẫn lệnh xuống thiết bị và tiết kiệm tài nguyên CPU/RAM trên thiết bị host nhúng.
Mô hình theo dõi vòng đời lệnh điều khiển (Command Lifecycle)	Cho phép theo dõi tường minh trạng thái xử lý gói tin qua các pha (accepted, queued, sent, executed, failed, timeout), giải quyết bài toán lệnh bị treo hoặc mất gói vô tuyến mà cloud không phát hiện được.
Chuẩn hóa mô hình nhận dạng thiết bị qua device_id logic	Tách biệt định danh cơ sở dữ liệu khỏi địa chỉ phần cứng cố định (EUI-64) hoặc địa chỉ động (16-bit nwk_addr), tăng tính bảo mật và cho phép linh hoạt thay thế phần cứng lỗi.
Bộ kịch bản kiểm thử E2E tích hợp evidence thực tế	Cung cấp phương pháp luận kiểm thử rõ ràng từ mức đơn vị đến tích hợp hệ thống, đi kèm các bằng chứng vận hành (log trace) chứng minh hệ thống hoạt động chính xác.

Các đóng góp trên được minh chứng thông qua kết quả kiểm thử thực tế tại Chương 5, cho thấy tính khả thi và độ tin cậy của giải pháp tự phát triển trong môi trường nghiên cứu phòng thí nghiệm.

1.6 Cấu trúc báo cáo

Nội dung báo cáo đề án tốt nghiệp được tổ chức thành 6 chương chính sau đây: **Chương 1 – Tổng quan đề tài:** Giới thiệu bối cảnh, lý do chọn đề tài, xác định mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Cơ sở lý thuyết và công nghệ nền: Trình bày nền tảng lý thuyết về giao thức

mạng Zigbee, thư viện cluster ZCL, giao thức MQTT, giao tiếp nối tiếp UART/ASH và các kit vi điều khiển phần cứng được sử dụng.

Chương 3 – Phân tích yêu cầu và thiết kế hệ thống: Phân tích các yêu cầu chức năng, phi chức năng, xây dựng sơ đồ kiến trúc tổng thể, mô hình dữ liệu, đặc tả giao diện API và hợp đồng trao đổi bản tin MQTT Contract.

Chương 4 – Triển khai hệ thống: Trình bày chi tiết quá trình lập trình firmware thiết bị, viết ứng dụng Gateway C, lập trình Cloud Backend bằng FastAPI và giải quyết các lỗi thực tế khi deploy lên EC2.

Chương 5 – Kiểm thử và đánh giá: Thiết lập kế hoạch kiểm thử, trình bày kết quả chạy các test case cụ thể, ghi nhận các bằng chứng logs/traces và phân tích ưu điểm cũng như hạn chế của hệ thống.

Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả đạt được, tự đánh giá năng lực tích lũy của bản thân và đề xuất các hướng nghiên cứu mở rộng trong tương lai như bảo mật TLS, cập nhật OTA và tích hợp WebSocket.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN

Chương này tập trung giải thích các nền tảng lý thuyết và công nghệ cốt lõi làm cơ sở cho việc thiết kế và triển khai hệ thống. Nội dung bao gồm tổng quan về IoT và kiến trúc tòa nhà thông minh, giao thức truyền thông mạng không dây Zigbee, thư viện ZCL định nghĩa mô hình thiết bị, giao thức nhắn tin MQTT, kênh giao tiếp UART và đặc tả EZSP/ASH, cùng thông tin chi tiết về phần cứng và kit phát triển EFR32MG12.

2.1 Tổng quan IoT và Smart Building

Internet vạn vật (Internet of Things - IoT) là mạng lưới các thiết bị vật lý được tích hợp cảm biến, phần mềm và các công nghệ kết nối khác nhằm mục đích trao đổi dữ liệu với các thiết bị và hệ thống khác qua Internet. Trong bối cảnh Smart Building (tòa nhà thông minh), hạ tầng IoT đóng vai trò xương sống kết nối hàng loạt cảm biến và cơ cấu chấp hành để thực hiện giám sát tự động.

Hệ thống điều khiển trong tòa nhà thông minh được tổ chức theo mô hình dòng dữ liệu khép kín bao gồm các pha: thu thập dữ liệu (sensing), truyền dẫn (communication), xử lý trung tâm (processing), ra quyết định điều khiển (control) và hiển thị giám sát (monitoring). Các cảm biến (nhiệt độ, ánh sáng, hiện diện) liên tục đo đạc môi trường vật lý, gửi dữ liệu qua mạng truyền thông không dây cục bộ tới bộ xử lý trung tâm (Gateway). Từ đó, Gateway đồng bộ trạng thái lên đám mây và nhận các quyết định điều khiển gửi xuống các cơ cấu chấp hành (đèn, rơ-le công tắc) để thay đổi trạng thái môi trường.

2.2 Giao thức truyền thông Zigbee

Để kết nối các thiết bị cảm biến và chấp hành có mật độ cao trong tòa nhà, giao thức mạng không dây tiêu thụ năng lượng thấp Zigbee được lựa chọn làm giải pháp truyền thông cục bộ chính.

2.2.1 Tổng quan về Zigbee

Zigbee là giao thức mạng không dây công suất thấp, tốc độ truyền dữ liệu thấp và độ tin cậy cao được xây dựng trên tiêu chuẩn IEEE 802.15.4 cho mạng cá nhân không dây (Wireless Personal Area Network - WPAN). Zigbee hoạt động chủ yếu trên dải tần không cấp phép ISM 2.4 GHz với tốc độ truyền dẫn lý thuyết là 250 kbps. Trong đề án này, hệ thống sử dụng phiên bản chuẩn hóa Zigbee 3.0 thông qua ngăn xếp EmberZNet (Gecko SDK 4.5.0) của hãng Silicon Labs, hỗ trợ mã hóa bảo mật AES-128 bit và cơ chế Trust Center để kiểm soát thiết bị gia nhập mạng một cách an toàn.

2.2.2 Vai trò thiết bị trong mạng Zigbee

Mạng truyền thông Zigbee định nghĩa ba vai trò thiết bị vật lý cơ bản để phân cấp quản lý: **Zigbee Coordinator (ZC):** Thiết bị duy nhất có nhiệm vụ khởi tạo mạng vô tuyến ban đầu, cấu hình các tham số bảo mật và đóng vai trò làm Trust Center cấp phát khóa mạng. Trong hệ thống này, Coordinator chạy firmware NCP (Network Co-Processor) kết nối trực tiếp với Gateway host qua cổng nối tiếp UART. Địa chỉ mạng 16-bit của Coordinator luôn cố định là 0x0000.

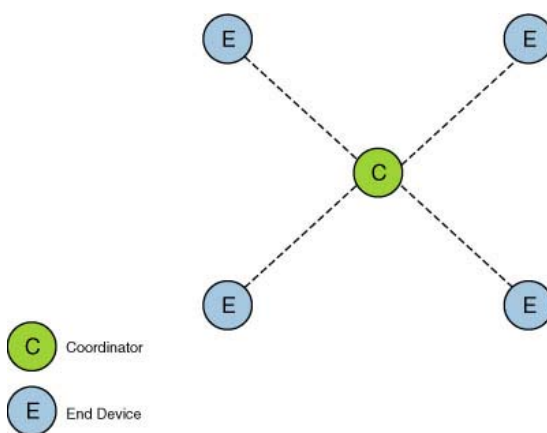
Zigbee Router (ZR): Thiết bị có nguồn cấp trực tiếp (không dùng pin), có nhiệm vụ định tuyến và chuyển tiếp các gói tin trong mạng mesh để mở rộng khoảng cách phủ sóng. Trong đồ án, các thiết bị đèn thông minh (Z3Light) và công tắc (Z3Switch) hoạt động ở vai trò Router, luôn luôn ở trạng thái thu phát sóng và không chuyển sang chế độ sleep.

Zigbee End Device (ZED): Thiết bị đầu cuối thu thập dữ liệu cảm biến, thường sử dụng nguồn pin và hoạt động ở chế độ Sleepy End Device (SED). Thiết bị SED có thể tắt khối vô tuyến thu phát để đi vào trạng thái ngủ sâu tiết kiệm pin khi không có sự kiện và chỉ thức dậy định kỳ để gửi dữ liệu tới thiết bị cha (Parent Node) của nó. Cảm biến hiện diện (Occupancy Sensor) trong đồ án được cấu hình ở vai trò ZED để tối ưu hóa năng lượng tiêu thụ.

2.2.3 Topology mạng vô tuyến

Zigbee hỗ trợ ba topo mạng chính bao gồm topo hình sao, topo hình cây và topo mạng lưới.

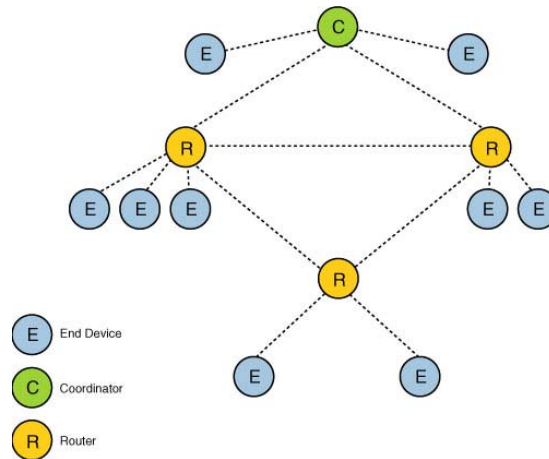
Mạng hình sao (Star Topology) được mô tả trong Hình 2.1, là mô hình đơn giản nhất trong đó tất cả các node Router và End Device kết nối trực tiếp đến một Coordinator trung tâm.



Hình 2.1: Star topology – Thiết bị kết nối qua Coordinator trung tâm

Ưu điểm của hình sao là dễ quản lý, trễ truyền dẫn nhỏ nhưng nhược điểm lớn là vùng phủ sóng bị giới hạn bởi công suất phát của một node Coordinator và Coordinator trở thành điểm lỗi duy nhất của toàn hệ thống (single point of failure).

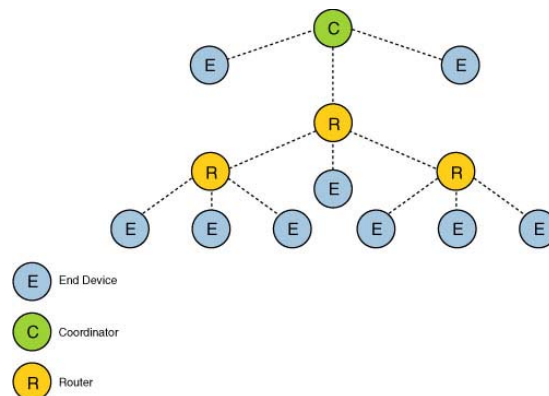
Mạng lưới (Mesh Topology) được thể hiện trong Hình 2.2, là cấu trúc mạnh mẽ nhất của chuẩn Zigbee. Các node Router có khả năng tự động liên kết nối mạng với nhau tạo thành mạng lưới truyền dẫn điểm-điểm đa chặng (multi-hop routing).



Hình 2.2: Mesh topology – Định tuyến đa chặng linh hoạt

Khi một chặng liên kết giữa hai node Router bị lỗi vật lý (do cản trở hoặc nhiễu sóng), thuật toán định tuyến AODV tích hợp sẵn trong NWK layer sẽ tự động tìm kiếm đường đi thay thế khác để truyền đạt thông tin tới đích, đảm bảo hệ thống tự phục hồi (self-healing).

Mạng hình cây (Tree Topology) được mô phỏng trong Hình 2.3, là cấu trúc phân cấp hình cây trong đó Coordinator là gốc, các Router đóng vai trò nhánh cây trung gian và End Device là lá cây ở cấp thấp nhất.

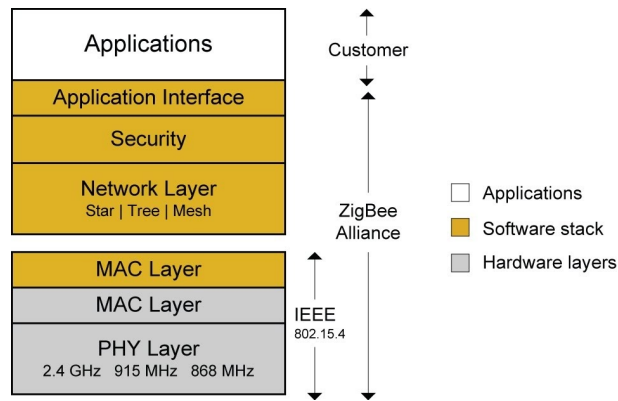


Hình 2.3: Tree topology – Cấu trúc phân cấp hình cây

Topo này kết hợp sự đơn giản trong định địa chỉ của topo hình sao và khả năng mở rộng khoảng cách của topo mạng lưới, phù hợp cho các mô hình quản lý phân lớp trong tòa nhà. Trong đồ án này, hệ thống triển khai thực tế theo cấu hình mạng mesh-tree linh hoạt.

2.2.4 Kiến trúc Protocol Stack

Kiến trúc phân tầng của giao thức Zigbee được xây dựng trên nền tảng chuẩn vật lý IEEE 802.15.4 như được mô tả chi tiết trong Hình 2.4.



Hình 2.4: Kiến trúc phân tầng Zigbee Protocol Stack

Sự phân chia chức năng giữa các lớp trong stack bao gồm: **Lớp PHY (Physical Layer):** Được định nghĩa bởi chuẩn IEEE 802.15.4, chịu trách nhiệm truyền phát các bit dữ liệu trên dải sóng vô tuyến tần số 2.4 GHz, thực hiện điều chế tín hiệu DSSS (Direct Sequence Spread Spectrum) và đo năng lượng kênh (LQI).

Lớp MAC (Medium Access Control): Quản lý việc truy cập kênh vô tuyến chung sử dụng cơ thức CSMA-CA (Carrier Sense Multiple Access with Collision Avoidance), kiểm soát lỗi truyền tin cậy qua cơ chế ACK và định dạng cấu trúc khung truyền MAC.

Lớp NWK (Network Layer): Do Zigbee Alliance định nghĩa, phụ trách cấp phát địa chỉ mạng 16-bit động cho thiết bị khi gia nhập mạng, xây dựng bảng định tuyến và bảo mật tầng mạng bằng Network Key dùng chung.

Lớp APS (Application Support Sublayer): Cầu nối trung gian giữa lớp mạng NWK và lớp ứng dụng ZCL. APS phụ trách lọc các bản tin trùng lặp, duy trì Bảng Binding (liên kết trực tiếp) và thực hiện phân mảnh bản tin nếu kích thước dữ liệu vượt quá giới hạn MTU (127 bytes).

Lớp ZCL (Zigbee Cluster Library): Định nghĩa mô hình trao đổi dữ liệu hướng đối tượng (cluster/attribute/command) giúp chuẩn hóa hành vi ứng dụng của thiết bị.

2.2.5 Địa chỉ và bảo mật trong mạng

Mỗi thiết bị Zigbee sở hữu hai loại địa chỉ phân biệt để phục vụ truyền thông: **Địa chỉ vật lý EUI-64 (Extended Unique Identifier):** Là địa chỉ MAC 64-bit cố định duy nhất được nhà sản xuất nạp sẵn vào chip bán dẫn. Địa chỉ này được đồ án sử dụng làm khóa định danh duy nhất (primary key) để xác định thiết bị trên MQTT và cơ sở dữ liệu Cloud, đảm bảo tính nhất quán ngay cả khi thiết bị ngắt kết nối vật lý.

Địa chỉ mạng 16-bit (NwkAddr): Là địa chỉ runtime được Coordinator cấp phát động cho thiết bị khi gia nhập mạng. Địa chỉ này có thể thay đổi sau mỗi lần thiết bị khởi động lại hoặc thực hiện rejoin mạng, do đó chỉ được dùng cho mục đích định tuyến lớp thấp của stack Zigbee và không được dùng làm primary key trên database.

Bảo mật mạng được kiểm soát thông qua hai loại khóa chính: Network Key 128-bit dùng để mã hóa toàn bộ dữ liệu truyền trên không trung ở tầng NWK và Link Key 128-bit dùng để mã hóa kênh truyền bảo mật điểm-điểm giữa Coordinator/Trust Center và một thiết bị cụ thể khi trao đổi Network Key.

2.2.6 Quá trình hình thành và gia nhập mạng

Mạng được hình thành khi Gateway ra lệnh cho Coordinator (chạy firmware NCP) khởi chạy quét năng lượng vô tuyến để tự động chọn kênh tần số ít nhiễu nhất trong số 16 kênh từ kênh 11 đến 26. Sau khi thiết lập mạng thành công với một ID mạng PAN ID ngẫu nhiên không

trùng lặp, Gateway ra lệnh mở cửa sổ permit-join trong một khoảng thời gian giới hạn (ví dụ 60 giây).

Quá trình thiết bị con (End Device hoặc Router) gia nhập mạng diễn ra qua bốn bước cụ thể sau: **Bước 1 – Khám phá mạng (Network Discovery):** Thiết bị con phát quảng bá tin nhắn Beacon Request để quét các mạng xung quanh. Coordinator nhận được tin nhắn và phản hồi lại bằng Beacon Frame chứa thông tin PAN ID và trạng thái cho phép gia nhập mạng (Association Permit = True).

Bước 2 – Kết nối vật lý (MAC Association): Thiết bị con gửi yêu cầu kết nối MAC Association Request chứa địa chỉ EUI-64 và khả năng phần cứng (node capability). Coordinator phê duyệt và cấp phát một địa chỉ mạng 16-bit ngẫu nhiên thông qua bản tin MAC Association Response.

Bước 3 – Xác thực bảo mật và trao đổi khóa: Coordinator (Trust Center) mã hóa Network Key hiện tại bằng Link Key mặc định (chẳng hạn Link Key tiêu chuẩn ZigbeeAlliance09) và gửi xuống thiết bị con qua bản tin Transport Key. Thiết bị con nhận, giải mã để lấy Network Key và lưu trữ.

Bước 4 – Cập nhật và Thông báo (Device Announce): Thiết bị con phát quảng bá bản tin Device Announce chứa địa chỉ EUI-64 và địa chỉ mạng 16-bit mới của nó để thông báo cho toàn mạng biết sự hiện diện của mình. Lúc này, Gateway sẽ bắt sự kiện này để cập nhật bảng lưu trữ cục bộ.

2.3 Thư viện cluster Zigbee Cluster Library (ZCL)

ZCL đóng vai trò là một chuẩn mô tả dữ liệu ở lớp ứng dụng, định nghĩa hành vi và định dạng trao đổi thông tin thống nhất giữa các thiết bị thuộc nhiều nhà sản xuất khác nhau.

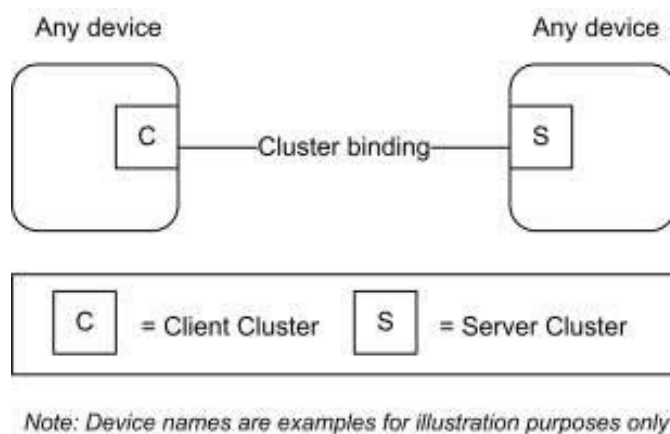
2.3.1 Cấu trúc mô hình ZCL

Mô hình ZCL tổ chức dữ liệu theo kiến trúc hướng đối tượng, bao gồm: **Cluster (Cụm chức năng):** Là đơn vị logic gom nhóm các thuộc tính và lệnh điều khiển có liên quan chặt chẽ tới một chức năng vật lý cụ thể (ví dụ: On/Off Cluster, Temperature Measurement Cluster). Mỗi cluster được gán một mã ID 16-bit duy nhất.

Attribute (Thuộc tính): Các biến lưu trữ trạng thái thực tế của thiết bị bên trong một cluster (ví dụ: thuộc tính On/Off kiểu Boolean lưu trạng thái bật/tắt của đèn, thuộc tính MeasuredValue lưu giá trị nhiệt độ đo được).

Command (Lệnh điều khiển): Các hành động tác động gửi tới thiết bị để thay đổi thuộc tính hoặc yêu cầu thiết bị thực thi một tác vụ (ví dụ: lệnh Toggle gửi tới On/Off cluster để đảo trạng thái thuộc tính On/Off).

ZCL phân chia rõ ràng vai trò Client và Server. Server cluster là nơi lưu giữ giá trị thực tế của các thuộc tính (ví dụ: bóng đèn chứa On/Off Server Cluster chứa biến trạng thái của rô-le đèn). Client cluster là nơi gửi đi các lệnh điều khiển thuộc tính hoặc truy vấn thuộc tính (ví dụ: công tắc chứa On/Off Client Cluster để gửi lệnh bật/tắt đi). Sự phối hợp Client/Server được mô phỏng trong Hình 2.5.



Hình 2.5: Mô hình tương tác Client/Server trong ZCL

2.3.2 Cơ chế Binding (Liên kết)

Binding là cơ chế cốt lõi của lớp APS cho phép thiết lập liên kết trực tiếp giữa hai endpoint của hai thiết bị Zigbee (ví dụ: liên kết từ On/Off Client của Switch tới On/Off Server của Light). Khi Bảng Binding được thiết lập thành công trên Coordinator, Switch có thể phát lệnh gửi đi trực tiếp thông qua địa chỉ logic của liên kết mà không cần ghi rõ địa chỉ 16-bit của Light. Cơ chế này giúp thiết bị công tắc bật/tắt đèn trực tiếp trong mạng cục bộ mà không cần sự can thiệp hay xử lý của phần mềm Gateway hoặc Cloud, giảm độ trễ tối đa và đảm bảo hệ thống vận hành tự động cục bộ khi mất kết nối mạng WAN.

2.4 Vai trò của Gateway trong hệ thống IoT

Gateway đóng vai trò là phần tử biên (Edge Device) chuyển đổi giao thức truyền thông hai chiều giữa lớp vô tuyến cục bộ RF và lớp mạng IP diện rộng.

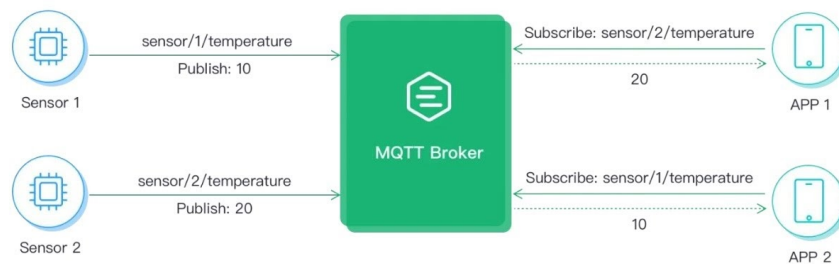
Gateway thu nhận các bản tin báo cáo thuộc tính ZCL (ZCL Attribute Reports) nhận từ thiết bị Zigbee qua UART/EZSP, phân tích dữ liệu nhị phân thô, sau đó chuyển đổi và đóng gói thành bản tin JSON tiêu chuẩn. JSON payload này được publish lên các MQTT topics tương ứng để Cloud xử lý. Ở chiều ngược lại, Gateway lắng nghe các yêu cầu điều khiển từ Cloud qua MQTT, giải mã và dispatch lệnh thành gói tin EZSP để Coordinator truyền xuống thiết bị đích.

2.5 Giao thức truyền thông MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông điệp gọn nhẹ hoạt động theo mô hình Publish/Subscribe chạy trên nền tảng TCP/IP.

2.5.1 Kiến trúc Publish/Subscribe

Mô hình publish/subscribe của MQTT loại bỏ hoàn toàn sự kết nối trực tiếp giữa các máy khách gửi dữ liệu (Publishers) và máy khách nhận dữ liệu (Subscribers) bằng cách sử dụng một máy chủ trung gian gọi là Broker (Mosquitto Broker). Khi một client publish tin nhắn tới một chủ đề cụ thể (Topic), Broker sẽ nhận bản tin đó và chịu trách nhiệm phân phối bản tin đến tất cả các clients đã đăng ký lắng nghe chủ đề đó như được thể hiện trong Hình 2.6.



Hình 2.6: Kiến trúc truyền thông Publish/Subscribe qua Broker trung gian

2.5.2 Quality of Service (QoS) trong MQTT

MQTT định nghĩa ba mức đảm bảo chất lượng dịch vụ (QoS) nhằm cân bằng giữa độ tin cậy truyền dẫn và băng thông đường truyền: **QoS 0 (At most once)**: Broker hoặc client gửi tin nhắn một lần duy nhất và không yêu cầu xác nhận. Tin nhắn có thể bị mất nếu mạng không ổn định, nhưng đây là mức nhanh nhất và tốn ít băng thông nhất. Phù hợp cho các bản tin telemetry báo cáo nhiệt độ định kỳ.

QoS 1 (At least once): Đảm bảo tin nhắn được truyền đến đích ít nhất một lần qua cơ chế xác nhận gói tin (PUBACK). Nếu bên gửi không nhận được PUBACK sau khoảng thời gian timeout, nó sẽ gửi lại tin nhắn (dẫn tới nguy cơ trùng lặp bản tin). Đề án sử dụng mức QoS 1 cho toàn bộ luồng command và state reported quan trọng.

QoS 2 (Exactly once): Đảm bảo tin nhắn đến đích đúng một lần duy nhất thông qua quá trình bắt tay bốn bước phức tạp. Đây là mức chậm nhất và tốn băng thông nhất, không được sử dụng trong hệ thống demo của đề án.

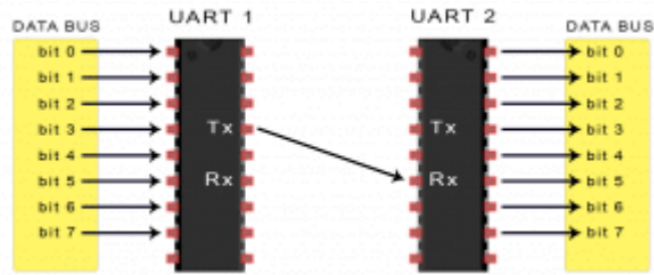
2.5.3 Retained Messages và Last Will and Testament (LWT)

Retained Message (Tin nhắn lưu trữ): Khi publish một bản tin với cờ Retain được set bằng True, Broker sẽ lưu trữ bản tin cuối cùng của topic đó. Khi một subscriber mới đăng ký topic đó, Broker sẽ lập tức gửi ngay bản tin lưu trữ này xuống, giúp subscriber mới cập nhật ngay trạng thái hiện tại của thiết bị mà không cần chờ thiết bị phát báo cáo chu kỳ tiếp theo.

Last Will and Testament (LWT - Di chúc): LWT cho phép một client khi kết nối với Broker đăng ký một bản tin mặc định cùng topic tương ứng. Nếu client bị ngắt kết nối đột ngột không mong muốn (ví dụ mất nguồn điện Gateway), Broker sẽ tự động publish bản tin "di chúc" này để thông báo cho hệ thống Cloud biết thiết bị Gateway đã đi vào trạng thái offline.

2.6 Giao tiếp nối tiếp UART

UART (Universal Asynchronous Receiver/Transmitter) là giao thức truyền thông nối tiếp bất đồng bộ, sử dụng hai đường dây tín hiệu TX (truyền) và RX (nhận) để thực hiện kết nối vật lý point-to-point trực tiếp giữa host Linux chạy Z3Gateway và chip NCP EFR32MG12 như trong Hình 2.7.



Hình 2.7: Kết nối UART vật lý giữa Host và NCP

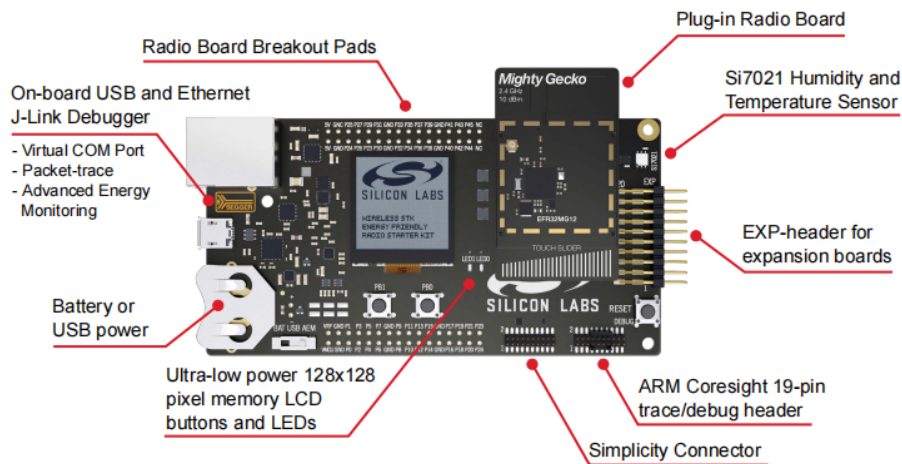
Giao tiếp UART bắt đồng bộ yêu cầu hai bên thống nhất cấu hình tốc độ truyền (Baud rate), bit dữ liệu, bit dừng (Stop bit) và bit kiểm tra chẵn lẻ (Parity). Trong đồ án, UART kết nối Host-NCP hoạt động ở baud rate mặc định 115200 bps, kết hợp giao thức ASH (Asynchronous Serial Host) do Silicon Labs đặc tả để đóng khung dữ liệu (framing), kiểm tra lỗi CRC và điều khiển luồng (flow control) nhằm bảo vệ tính toàn vẹn của các khung lệnh EZSP.

2.7 Phần cứng và Kit phát triển sử dụng

Để triển khai các node thiết bị thực tế, đồ án sử dụng bộ công cụ phát triển phần cứng chuyên nghiệp của hãng Silicon Labs.

2.7.1 Wireless Starter Kit BRD4001A và BRD4162A

Thiết bị phần cứng cốt lõi được sử dụng là kit phát triển EFR32MG12 2.4 GHz 10 dBm Wireless Starter Kit (WSTK) như được chụp lại trong Hình 2.8.



Hình 2.8: Hình ảnh bộ kit phát triển EFR32MG12 Wireless Starter Kit

Bộ kit phát triển bao gồm hai thành phần bo mạch lắp ghép chính: **Radio Board BRD4162A:** Là bo mạch cắm chứa SoC EFR32MG12P332F1024GL125 chuyên dụng cho truyền thông không dây đa giao thức (Wireless Gecko). Chip bán dẫn này tích hợp lõi vi xử lý ARM Cortex-M4 32-bit tốc độ 40 MHz, dung lượng bộ nhớ Flash 1024 kB phục vụ lưu trữ firmware, bộ nhớ RAM 256 kB và mạch thu phát vô tuyến công suất phát tối đa +10 dBm trên băng tần 2.4 GHz kèm anten mạch in PCB anten tích hợp.

Mainboard BRD4001A: Là bo mạch mẹ cung cấp mạch nạp Segger J-Link tích hợp để gỡ lỗi mã nguồn, màn hình LCD hiển thị thông tin, cảm biến đo nhiệt độ Si7021, cùng các nút

nhấn vật lý (BTN0, BTN1) và đèn LED (LED0, LED1) để lập trình mô phỏng thiết bị.

2.7.2 Cấu hình phần cứng theo vai trò ứng dụng

Các board phần cứng EFR32MG12 được lập trình nạp firmware riêng biệt để mô phỏng bốn thành phần vật lý của mạng Zigbee: **Coordinator/NCP Board:** Cắm trực tiếp vào host qua cổng USB để thiết lập kết nối VCOM UART ảo, chạy firmware NCP thuần để làm bộ thu phát sóng Zigbee cho gateway.

Z3Light Board: Sử dụng LED0 trên Mainboard để mô phỏng trạng thái bóng đèn (LED0 sáng tương ứng đèn ON, LED0 tắt tương ứng đèn OFF).

Z3Switch Board: Sử dụng nút nhấn BTN0 trên Mainboard để gửi lệnh Toggle thay đổi trạng thái của đèn qua binding.

Occupancy Sensor Board: Cấu hình ở chế độ Sleepy End Device chạy nguồn pin, sử dụng nút nhấn BTN0 để mô phỏng sự kiện cảm biến phát hiện có người di chuyển.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Chương này tập trung trình bày các kết quả phân tích yêu cầu chức năng, phi chức năng và thiết kế kiến trúc cho toàn bộ hệ thống Smart Building Platform. Nội dung chi tiết bao gồm mô hình kiến trúc phân lớp, các luồng dữ liệu Uplink/Downlink, thiết kế mô hình dữ liệu thiết bị, thiết kế cấu trúc cơ sở dữ liệu, đặc tả giao diện REST API và đặc tả hợp đồng bản tin MQTT Contract, cùng phân tích các quyết định trade-off thiết kế quan trọng.

3.1 Phân tích yêu cầu hệ thống

Để xây dựng một hệ thống IoT hoàn chỉnh hoạt động ổn định và có thể mở rộng, trước tiên cần phân tích các yêu cầu chức năng và phi chức năng cốt lõi.

3.1.1 Yêu cầu chức năng

Hệ thống được thiết kế để đáp ứng các yêu cầu chức năng chính được phân lớp rõ ràng, tóm tắt trong Bảng 3.1.

Bảng 3.1: Các yêu cầu chức năng chính của hệ thống

Nhóm tính năng	Yêu cầu chức năng	Mô tả chi tiết
Quản lý thiết bị	Join mạng thiết bị	Cho phép thiết bị Zigbee mới gia nhập vào mạng thông qua việc kích hoạt mở permit-join từ xa.
	Quản lý danh mục	Quản lý thông tin tổ chức tòa nhà (nhà, phòng) và danh sách thiết bị logic qua REST API.
Giám sát & Điều khiển	Báo cáo trạng thái	Cập nhật và lưu trữ trạng thái thực tế (reported state) của thiết bị lên Cloud khi có thay đổi.
	Điều khiển phụ tải	Cho phép gửi lệnh bật/tắt thiết bị đèn từ Cloud xuống mạng Zigbee thông qua Gateway.
Tự động hóa	Liên kết trực tiếp	Cho phép công tắc điều khiển đèn cục bộ thông qua cơ chế binding lớp vô tuyến của Zigbee.
	Theo dõi vòng đời lệnh	Theo dõi trạng thái lệnh qua các pha (accepted, queued, sent, executed, failed, timeout).

Chi tiết các yêu cầu chức năng trong Bảng 3.1 làm định hướng để thiết kế cấu trúc cơ sở dữ liệu và các API endpoints tương ứng tại các mục tiếp theo.

3.1.2 Yêu cầu phi chức năng

Hệ thống phải đảm bảo các tiêu chí kỹ thuật vận hành phi chức năng sau: **Độ trễ phản hồi (Latency):** Thời gian xử lý lệnh từ lúc client gọi API gửi lệnh điều khiển cho tới khi nhận

được phản hồi trạng thái từ thiết bị đèn thực tế qua Gateway phải nằm trong ngưỡng cho phép (dưới 2 giây trong điều kiện mạng ổn định).

Khả năng hoạt động độc lập (Offline Capability): Khi mất kết nối internet lên Cloud Backend, các kịch bản tự động hóa cục bộ (công tắc bật/tắt đèn qua local binding) phải hoạt động bình thường nhờ logic xử lý trọn vẹn trong mạng mesh Zigbee.

Giới hạn thời gian (Timeout): Thời gian chờ phản hồi tối đa của lệnh điều khiển thiết bị được thiết lập cố định là 5000 ms để tránh tình trạng lệnh bị treo vô hạn trong cơ sở dữ liệu.

Bảo mật phân quyền (Least Privilege): Broker Mosquitto phải cấu hình kiểm soát truy cập (ACL) phân tách rõ ràng quyền đọc/ghi của các tài khoản (gateway, client, monitor) để tránh lộ thông tin và can thiệp lệnh trái phép.

3.2 Kiến trúc tổng thể hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc phân lớp hướng dịch vụ để phân tách rõ ràng trách nhiệm của từng thành phần. Hình 3.1 mô tả cấu trúc 4 lớp chính của hệ thống.

Hình 3.1: Sơ đồ kiến trúc tổng quan hệ thống Smart Building

Kiến trúc hệ thống bao gồm bốn lớp vật lý chính phối hợp hoạt động:

Lớp Thiết bị (Device Layer): Gồm các nút thiết bị Zigbee 3.0 (Z3Light, Z3Switch, Occupancy Sensor) giao tiếp không dây với Coordinator.

Lớp Gateway trung gian (Gateway Layer): Gồm phần cứng host nhúng chạy hệ điều hành Linux và ứng dụng native C Z3Gateway. Ứng dụng này sử dụng chip NCP qua UART/ASH để điều khiển mạng Zigbee, đồng thời kết nối trực tiếp lên MQTT Broker. Bảng 3.2 liệt kê vai trò của các module mã nguồn tự thiết kế trong Z3Gateway.

Lớp Trung chuyển (Message Broker): Mosquitto MQTT Broker làm trung gian điều phối bản tin, hỗ trợ ACL bảo mật và cơ chế cầu nối (bridge) truyền thông.

Lớp Dịch vụ đám mây (Cloud Layer): Chứa dịch vụ Cloud Backend xây dựng trên framework FastAPI kết nối cơ sở dữ liệu PostgreSQL để lưu trữ trạng thái lâu bền và expose hệ thống REST API phục vụ người dùng.

Bảng 3.2: Các module logic chính trong ứng dụng Z3Gateway C

Tên module	Vai trò và nhiệm vụ kỹ thuật
app_mqtt.c/h	Khởi tạo client MQTT kết nối Broker, xử lý đăng ký nhận và phát bản tin.
cmd_handler.c/h	Phân tích cấu trúc envelope JSON của lệnh, dispatch lệnh tới module phần cứng.
device_registry.c/h	Quản lý bảng lưu trữ thông tin thiết bị trong bộ nhớ RAM tạm thời (eui64, 16-bit nwk_addr).
light_ctrl.c/h	Tạo các khung tin nhắn ZCL On/Off gửi xuống Coordinator để bật/tắt đèn.
telemetry_rx.c/h	Bắt các callback báo cáo thuộc tính ZCL từ NCP, chuyển đổi thành JSON reported.
net_mgr.c/h	Quản lý quá trình khởi tạo mạng Zigbee và mở cửa sổ cho phép thiết bị join.

Sự phân chia rõ ràng các module trong Bảng 3.2 đảm bảo tính mô-đun hóa cao, giúp Z3Gateway chạy ổn định dưới dạng một single-process C gọn nhẹ.

3.3 Thiết kế mạng và Vai trò thiết bị

Mạng Zigbee cục bộ sử dụng mô hình mesh-tree phân cấp. Coordinator đóng vai trò Trust Center duy nhất để quản lý bảo mật mạng, các node Z3Light và Z3Switch hoạt động dưới vai trò Router giúp chuyển tiếp gói tin, cảm biến chuyển động hoạt động dưới vai trò End Device tiết kiệm pin. Bảng 3.3 mô tả chi tiết sự ánh xạ giữa loại thiết bị và các cluster ZCL tương ứng được cấu hình.

Bảng 3.3: Bảng ánh xạ device type và các cluster ZCL cấu hình

Loại thiết bị	Zigbee Role	Server Clusters	Client Clusters
Coordinator / NCP	ZC (Coordinator)	–	On/Off Client (0x0006)
Light Device	ZR (Router)	On/Off (0x0006)	–
Switch Device	ZR (Router)	–	On/Off (0x0006)
Occupancy Sensor	ZED (End Device)	Occupancy Sensing (0x0406)	–

Cấu hình cluster trong Bảng 3.3 đảm bảo các thiết bị chỉ implement các chức năng tối thiểu theo đúng thiết kế của Silicon Labs để tiết kiệm bộ nhớ flash của chip.

3.4 Luồng dữ liệu chính trong hệ thống

Hệ thống được thiết kế dựa trên hai luồng truyền dẫn dữ liệu hai chiều Uplink và Downlink cốt lõi, cùng luồng tự động hóa cục bộ độc lập.

3.4.1 Luồng báo cáo trạng thái (Uplink Flow)

Uplink flow đảm nhận việc đồng bộ trạng thái thực tế của thiết bị vật lý lên cơ sở dữ liệu Cloud gần thời gian thực. Hình 3.2 mô tả chi tiết đường đi của dòng dữ liệu từ cảm biến lên giao diện giám sát.

Hình 3.2: Sơ đồ luồng dữ liệu Uplink đồng bộ trạng thái

Quá trình truyền dẫn Uplink diễn ra theo các bước tuần tự sau:

Bước 1 – Gửi báo cáo từ thiết bị: Thiết bị Zigbee (Light/Sensor) gửi bản tin báo cáo thuộc tính ZCL Attribute Report lên Coordinator.

Bước 2 – Nhận và chuyển đổi tại Coordinator: Coordinator nhận được gói tin vô tuyến, chuyển đổi thành khung tin nhắn EZSP truyền qua UART/ASH nối tiếp lên Host Gateway.

Bước 3 – Xử lý và publish tại Gateway: Module `telemetry_rx.c` trên Z3Gateway bắt callback dữ liệu, parse giá trị nhị phân thành định dạng trạng thái tương ứng, sau đó chuyển cho module `app_mqtt.c` để publish tin nhắn reported lên MQTT Broker.

Bước 4 – Đăng ký và lưu trữ tại Cloud: Cloud Backend subscribe chủ đề reported qua MQTT client chạy nền, bắt được tin nhắn và lưu trữ giá trị trạng thái mới vào PostgreSQL.

3.4.2 Luồng gửi lệnh điều khiển (Downlink Flow)

Downlink flow chịu trách nhiệm truyền tải lệnh điều khiển bật/tắt từ người dùng xuống thiết bị chấp hành cuối. Hình 3.3 thể hiện chi tiết quá trình này.

Hình 3.3: Sơ đồ luồng điều khiển Downlink từ Cloud xuống thiết bị

Các bước cụ thể trong quy trình Downlink bao gồm:

Bước 1 – Gửi yêu cầu điều khiển từ UI: Người dùng thao tác trên giao diện, gọi REST API điều khiển (POST /api/devices/{id}/command).

Bước 2 – Tiếp nhận và gửi request từ Cloud: Cloud API tạo bản ghi lệnh điều khiển mới trong database với trạng thái khởi tạo accepted, đồng thời publish tin nhắn command request lên MQTT topic của Gateway.

Bước 3 – Nhận và phân tích lệnh tại Gateway: Gateway nhận được bản tin MQTT qua module `app_mqtt.c`, tiến hành parse JSON envelope tại `cmd_handler.c` để xác minh tính hợp lệ.

Bước 4 – Tra cứu địa chỉ và gửi lệnh ZCL: Gateway tra cứu bảng địa chỉ thiết bị trong RAM tại `device_registry.c` để lấy địa chỉ 16-bit tương ứng của Light, sau đó gọi `light_ctrl.c` gửi khung lệnh ZCL On/Off xuống Coordinator qua UART.

Bước 5 – Thực thi lệnh và phản hồi: Thiết bị Light thực thi lệnh vật lý, đổi trạng thái LED và gửi trả bản tin phản hồi ngược lại Gateway để báo cáo lệnh đã chạy thành công.

3.5 Thiết kế dữ liệu và Hợp đồng giao tiếp (MQTT Contract)

Để đảm bảo các tiến trình viết bằng các ngôn ngữ khác nhau (C trên Gateway, Python trên Cloud) có thể trao đổi thông tin chính xác, hệ thống định nghĩa một bộ hợp đồng giao tiếp chuẩn hóa duy nhất dựa trên JSON Schema.

3.5.1 Phân cấp cấu trúc Topics

Hệ thống sử dụng cấu trúc topic phân cấp hỗ trợ đa khách hàng (multi-tenant) và đa địa điểm (multi-site):

```
sb/v1/{tenant_id}/{site_id}/{gateway_id}/{channel_type}/...
```

Trong đó, các giá trị mặc định được cấu hình trong môi trường thử nghiệm bao gồm: `tenant_id = hust`, `site_id = lab01`, và `gateway_id = gw-ubuntu-01`. Bảng 3.4 mô tả các nhóm kênh truyền MQTT được thiết kế.

Bảng 3.4: Thiết kế phân lớp các kênh truyền MQTT

Nhóm kênh	Hướng truyền	Publisher	Subscriber	QoS
gateway/online	Uplink	Gateway	Cloud	1 (Retain)
gateway/health	Uplink	Gateway	Cloud	0
reported	Uplink	Gateway	Cloud	1 (Retain)
command	Downlink	Cloud	Gateway	1
reply	Uplink	Gateway	Cloud	1

Việc cấu hình QoS 1 cho các kênh `command` và `reply` trong Bảng 3.4 đảm bảo lệnh điều khiển không bị mất mát khi đường truyền mạng WAN gặp sự cố ngắn hạn.

3.5.2 Cấu trúc Envelope bản tin chung

Mọi tin nhắn trao đổi qua lại giữa Gateway và Cloud đều bắt buộc sử dụng một envelope JSON chung chứa siêu dữ liệu định danh, giúp thống nhất việc deserialization:

Mã nguồn 3.1: Cấu trúc envelope JSON chung của bản tin

```
{
  "schema": "sb.v1",
  "msg_id": "uuid-cua-ban-tin",
  "ts": "ISO8601-timestamp",
  "tenant_id": "hust",
  "site_id": "lab01",
  "gateway_id": "gw-ubuntu-01",
  "source": "gateway|cloud",
  "correlation_id": "uuid-lien-ket-optional",
  "payload": {}
}
```

Trường `correlation_id` được sử dụng để gắn kết kết quả phản hồi trên kênh `reply` với mã lệnh điều khiển gốc gửi đi từ kênh `command`, giúp Cloud theo dõi chính xác tiến trình xử lý lệnh.

3.6 Thiết kế cơ sở dữ liệu và Cloud Backend

Dịch vụ đám mây xử lý lưu trữ trạng thái lâu bền và expose giao diện REST API.

3.6.1 Thiết kế cơ sở dữ liệu quan hệ (PostgreSQL)

Cấu trúc cơ sở dữ liệu quan hệ PostgreSQL gồm 7 bảng dữ liệu chính để quản lý tổ chức hệ thống, được tóm tắt trong Bảng 3.5.

Bảng 3.5: Các bảng cơ sở dữ liệu chính của Cloud Backend

Tên bảng	Vai trò nhiệm vụ lưu trữ
homes	Lưu thông tin tòa nhà/khu vực lớn nhất.
rooms	Lưu thông tin phân chia các phòng thuộc tòa nhà.
users	Quản lý thông tin tài khoản người dùng hệ thống.
devices	Quản lý danh mục thiết bị logic (lưu <code>device_id</code> logic và EUI-64).
device_states	Lưu trữ ảnh trạng thái hiện tại (reported state) dạng JSON.
events	Lưu log lịch sử các sự kiện xảy ra trong hệ thống.
commands	Theo dõi trạng thái chi tiết của lệnh điều khiển trong vòng đời.

3.6.2 Thiết kế REST API Endpoints

Hệ thống Cloud Backend expose các endpoints API theo tiêu chuẩn RESTful, sử dụng định dạng JSON làm chuẩn trao đổi thông tin. Bảng 3.6 liệt kê các REST API cốt lõi được thiết kế phục vụ ứng dụng.

Bảng 3.6: Đặc tả các REST API endpoints chính của hệ thống

Method	Path	Ý nghĩa chức năng
GET	/health	Kiểm tra trạng thái hoạt động của Cloud API.
GET	/api/devices	Liệt kê danh sách các thiết bị trong hệ thống.
GET	/api/devices/{device_id}/state	Truy vấn trạng thái reported mới nhất của thiết bị.
POST	/api/devices/{device_id}/command	Tạo lệnh điều khiển mới, gửi command xuống Gateway.
GET	/api/commands/{command_id}	Tra cứu trạng thái xử lý hiện tại của lệnh điều khiển.
GET	/api/events	Truy vấn lịch sử sự kiện (phân trang và lọc theo thiết bị).

3.7 Các quyết định thiết kế và Trade-offs quan trọng

Trong quá trình thiết kế hệ thống, một số quyết định kỹ thuật quan trọng đã được đưa ra để tối ưu hiệu năng: **Loại bỏ hoàn toàn cầu nối Python trung gian tại Gateway (Tháng 4/2026):** Kiến trúc cũ sử dụng một script Python chạy nền để làm bridge chuyển đổi IPC (Unix socket/NDJSON) sang MQTT. Quyết định tích hợp thư viện MQTT trực tiếp vào mã nguồn C của Z3Gateway giúp loại bỏ một tiến trình, tiết kiệm tài nguyên CPU nhúng và giảm trễ truyền dẫn. Tuy nhiên, đánh đổi lại là việc duy trì và thay đổi cấu trúc dữ liệu trên Gateway C đòi hỏi biên dịch lại mã nguồn C phức tạp hơn.

Sử dụng định danh logic device_id thay vì EUI-64 làm khóa bền: Địa chỉ EUI-64 gắn chặt với chip vô tuyến vật lý. Thiết kế sử dụng device_id logic tự sinh làm khóa chính trên Cloud giúp hệ thống dễ dàng thay thế phần cứng thiết bị bị hỏng bằng một thiết bị mới mà không cần cập nhật lại toàn bộ các liên kết trong database.

Chỉ Retain bản tin State, không Retain bản tin Command: Việc retain bản tin trạng thái reported giúp các client mobile khi kết nối sau lấy ngay được giao diện cập nhật. Đối với command, không sử dụng retain để tránh tình trạng khi Gateway mất kết nối và online trở lại bị Broker gửi lại các lệnh bật/tắt cũ, gây nguy hiểm cho thiết bị chấp hành thực tế.

CHƯƠNG 4

TRIỂN KHAI HỆ THỐNG

Chương này mô tả chi tiết quá trình lập trình triển khai mã nguồn thực tế cho các thành phần trong hệ thống. Nội dung bao gồm triển khai firmware thiết bị Zigbee trên kit EFR32MG12, lập trình Gateway Service C tích hợp MQTT trực tiếp, cấu hình và triển khai Cloud API sử dụng FastAPI đóng gói Docker trên AWS EC2, giải quyết các lỗi triển khai thực tiễn, phân tích hoạt động của luồng điều khiển end-to-end và đề xuất thiết kế hướng phát triển tính năng OTA.

4.1 Triển khai Firmware cho thiết bị Zigbee

Firmware cho các node thiết bị Zigbee 3.0 được lập trình bằng ngôn ngữ C sử dụng IDE Simplicity Studio phiên bản 5 kết hợp ngăn xếp EmberZNet thuộc Gecko SDK phiên bản 4.5.0.

4.1.1 Triển khai Coordinator / NCP Board

Firmware NCP (Network Co-Processor) được biên dịch cho kit BRD4162A đóng vai trò Coordinator. Firmware này không chứa logic ứng dụng smart home mà chỉ hoạt động như một card vô tuyến RF phối bày các dịch vụ EZSP qua UART. Khi khởi động, firmware khởi tạo lớp PHY/MAC trên dải tần 2.4 GHz, thiết lập Trust Center phục vụ trao đổi mã khóa an toàn và liên tục lắng nghe/gửi các gói tin Zigbee theo lệnh điều khiển từ Host Gateway nhúng truyền xuống qua UART.

4.1.2 Triển khai Router đèn (Z3Light Device)

Thiết bị đèn thông minh Z3Light được cấu hình hoạt động ở vai trò Router vô tuyến Zigbee, implement cụm chức năng On/Off Server Cluster (0x0006). Khi nhận được ZCL command yêu cầu bật hoặc tắt, callback của stack EmberZNet sẽ kích hoạt thay đổi mức logic trên chân GPIO điều khiển LED0 trên kit phần cứng:

Mã nguồn 4.1: Snippet xử lý thay đổi trạng thái LED trên Z3Light

```
void emberAfOnOffClusterServerAttributeChangedCallback(uint8_t endpoint,
                                                         EmberAfAttributeId
                                                         attributeId)
{
    if (attributeId == ATTRIBUTE_FREE_ATTRIBUTE_ID) {
        bool onOffState;
        emberAfReadServerAttribute(endpoint,
                                    ZCL_ON_OFF_CLUSTER_ID,
                                    ZCL_ON_OFF_ATTRIBUTE_ID,
                                    (uint8_t *)&onOffState,
                                    sizeof(onOffState));

        if (onOffState) {
            halSetLed(0); // LED0 ON
        }
    }
}
```

```

    } else {
        halClearLed(0); // LED0 OFF
    }
}
}

```

Đồng thời, sau khi thay đổi trạng thái thuộc tính OnOff, firmware tự động kích hoạt gửi bản tin ZCL Attribute Report lên Coordinator để cập nhật trạng thái mới nhất về hệ thống điều khiển trung tâm.

4.1.3 Triển khai Router công tắc (Z3Switch Device)

Thiết bị Z3Switch được cấu hình ở vai trò Router, implement cụm chức năng On/Off Client Cluster. Khi người dùng nhấn nút nhấn BTN0 trên bo mạch, firmware sẽ kích hoạt gửi đi bản tin ZCL Toggle Command trực tiếp tới địa chỉ endpoint đã được lưu trong Bảng Binding cục bộ mà không cần gửi gói tin qua Gateway. BTN1 được cấu hình để kích hoạt tính năng Network Steering giúp thiết bị tự động quét các mạng lân cận để gia nhập mạng.

4.1.4 Triển khai cảm biến hiện diện (Occupancy Sensor)

Cảm biến hiện diện được triển khai ở vai trò Sleepy End Device, sử dụng cụm chức năng Occupancy Sensing Server Cluster (0x0406). Để tiết kiệm năng lượng, firmware đi vào trạng thái ngủ sâu (sleep mode) tắt khối RF. Khi nút nhấn BTN0 được nhấn (mô phỏng cảm biến phát hiện chuyển động), một ngắt ngoài GPIO sẽ đánh thức vi điều khiển. Firmware lập tức thay đổi thuộc tính Occupancy thành True, gửi bản tin Attribute Report lên Parent Node để chuyển về Gateway, sau đó tự động thiết lập một bộ hẹn giờ tự tắt (vacancy timeout) để chuyển thuộc tính về False trước khi quay lại trạng thái ngủ sâu.

4.2 Triển khai Gateway Service nhúng

Ứng dụng Gateway được lập trình bằng ngôn ngữ C, biên dịch chéo cho môi trường Linux sử dụng tệp Makefile tự sinh từ Simplicity Studio.

4.2.1 Kiến trúc vận hành C-MQTT trực tiếp

Z3Gateway hoạt động như một tiến trình đơn (single-process) tích hợp trực tiếp thư viện Eclipse Paho MQTT C. Tiến trình Gateway chạy một vòng lặp sự kiện chính (event loop) để liên tục thăm dò dữ liệu nối tiếp từ cổng UART kết nối với NCP qua giao thức ASH/EZSP, đồng thời lắng nghe các bản tin TCP/IP từ socket kết nối lên MQTT Broker. Việc tích hợp direct MQTT client giúp loại bỏ hoàn toàn cầu nối Python và Unix socket IPC cũ, giải quyết triệt để các lỗi đồng bộ tiến trình và trễ hàng đợi.

4.2.2 Cơ chế ánh xạ Correlation ID

Khi Gateway nhận được một bản tin yêu cầu điều khiển từ kênh command, module cmd_handler.c sẽ bóc tách trường msg_id (UUID tự sinh từ Cloud) lưu vào bộ nhớ RAM tạm thời của Gateway. Khi gửi lệnh điều khiển ZCL xuống thiết bị thành công hoặc thất bại, Gateway sẽ publish bản tin phản hồi lên kênh reply với trường correlation_id được set bằng đúng giá trị msg_id đã lưu trước đó. Cơ chế này giúp Cloud Backend ánh xạ chính xác kết quả thực thi ở

biên vật lý về đúng yêu cầu API gốc của người dùng.

4.2.3 Triển khai Rules Engine cục bộ

Mã nguồn Gateway bao gồm các module `rule_engine.c` và `sb_command.c` chịu trách nhiệm xử lý các kịch bản tự động hóa cục bộ ở biên. Tuy nhiên, trong phiên bản hiện tại v1, logic xử lý của các file này mới dừng lại ở mức thiết lập khung định nghĩa cấu trúc dữ liệu cấu hình rule, việc đánh giá so sánh động các điều kiện rule tự chọn vẫn đang ở giai đoạn phát triển và sẽ được hoàn thiện trong tương lai.

4.3 Triển khai Cloud Backend và quy trình Deployment

Cloud Backend được xây dựng bằng Python 3.12 sử dụng framework FastAPI phiên bản 0.115.6, kết nối PostgreSQL phiên bản 16 qua thư viện SQLAlchemy 2.0.36 hỗ trợ chế độ kết nối bất đồng bộ `asyncpg`.

4.3.1 Cấu hình biến môi trường

Hệ thống Cloud Backend đọc toàn bộ cấu hình vận hành từ biến môi trường thông qua thư viện `Pydantic Settings` với tiền tố `SB_`. Bảng 4.1 liệt kê các biến cấu hình hệ thống chính.

Bảng 4.1: Các biến môi trường cấu hình chính của Cloud Backend

Biến cấu hình	Giá trị mặc định / Ý nghĩa
<code>SB_DATABASE_URL</code>	<code>postgresql+asyncpg://sb_user:sb_pass@localhost:5432/sb_cloud</code>
<code>SB_MQTT_HOST</code>	Địa chỉ IP hoặc tên miền của Mosquitto Broker.
<code>SB_MQTT_PORT</code>	Port kết nối MQTT Broker (mặc định 1883).
<code>SB_COMMAND_TIMEOUT_MS</code>	Ngưỡng thời gian sweep timeout lệnh điều khiển (5000 ms).
<code>SB_TENANT_ID</code> , <code>SB_SITE_ID</code>	Xác định namespace mặc định (hust, lab01).

4.3.2 Triển khai đồng bộ trạng thái và quét Timeout

Được triển khai trong tệp `cloud/app/mqtt_client.py`, client MQTT chạy bất đồng bộ kết nối vào Mosquitto Broker. Khi bắt được bản tin `reported state` của thiết bị từ Gateway gửi lên, module thực hiện câu lệnh `upsert` (update hoặc insert) ghi đè vào bảng `device_states` theo khóa `device_id` logic.

Đồng thời, một luồng background worker chạy định kỳ được định nghĩa trong `cloud/app/command_timeout.py` sẽ thực hiện quét bảng `commands` mỗi 1000 ms. Bản ghi lệnh nào có trạng thái là `sent` hoặc `accepted` có thời gian tạo vượt quá giá trị `SB_COMMAND_TIMEOUT_MS` mà chưa nhận được reply tương ứng sẽ tự động cập nhật trạng thái thành `timeout` để báo cho ứng dụng.

4.3.3 Quy trình Deploy thực tế trên AWS EC2

Để đưa hệ thống lên môi trường AWS EC2 production chạy thực tế, toàn bộ stack Cloud Backend được đóng gói dưới dạng các container Docker qua tệp `deploy/docker-compose.prod.yml`. Quá trình deploy được tự động hóa bằng bộ script PowerShell chạy từ máy tính Windows nạp mã nguồn lên Ubuntu Server trên EC2. Các lỗi kỹ thuật phát sinh thực tế và giải pháp khắc phục trong quá trình deploy bao gồm: **Lỗi ký tự CRLF của Windows:** Khi copy script từ Windows

lên Linux Server qua câu lệnh here-string của PowerShell, các ký tự xuống dòng `\n` khiến trình biên dịch bash của Linux báo lỗi cú pháp. Giải pháp là thực hiện loại bỏ ký tự `\n` bằng regex trước khi đẩy file đi.

Lỗi mã hóa BOM UTF-8: Lệnh Set-Content mặc định của PowerShell tự động thêm ký tự BOM (Byte Order Mark) ở đầu file gây lỗi đọc config trên Docker Linux. Giải pháp là cấu hình ép kiểu mã hóa UTF-8 không BOM thông qua thư viện UTF8Encoding \$false của .NET.

Xung đột Port 1883 của Mosquitto: Trên Ubuntu Server của EC2 có một tiến trình mosquitto cài đặt native tự khởi chạy chiếm dụng port 1883, dẫn tới container Docker sb-mosquitto không khởi động được. Giải pháp là chạy script tắt và vô hiệu hóa service native qua lệnh:

```
systemctl stop mosquitto
systemctl disable mosquitto
```

Quyền cấu hình kiểm soát ACL Broker: Broker yêu cầu bật xác thực bảo mật (`allow_anonymous false`). Script kiểm tra sức khỏe hệ thống cần sử dụng tài khoản monitor mật khẩu monitor123 đăng ký trong tệp cấu hình Mosquitto và được cấp quyền đọc chủ đề hệ thống (`topic read $SYS/#`).

4.4 Luồng vận hành thực tế End-to-End Command

Quy trình vận hành truyền lệnh điều khiển thực tế từ người dùng xuống thiết bị phần cứng qua hệ thống đã triển khai diễn ra như sau: **Bước 1 – Tạo lệnh và gửi lên Broker:** Người dùng gọi API điều khiển. Bản ghi lệnh điều khiển được lưu vào database và chuyển sang trạng thái `accepted`. Dịch vụ FastAPI publish một bản tin MQTT chứa payload JSON mô tả hành động điều khiển có cờ QoS = 1 lên topic `command`.

Bước 2 – Gateway nhúng nhận và phản hồi lệnh: Gateway nhúng Z3Gateway nhận bản tin từ Broker MQTT, publish phản hồi trạng thái `queued` và `sent` ngược lại MQTT báo lệnh đã bắt đầu xử lý biên. Module `cmd_handler.c` trên Gateway gọi `light_ctrl.c` để đóng gói lệnh ZCL chuyển xuống Coordinator qua cổng VCOM UART kết nối VCOM.

Bước 3 – Thiết bị chấp hành thực thi lệnh: Thiết bị đèn Z3Light nhận được khung tin vô tuyến, thay đổi trạng thái của LED0, đồng thời tự phát Attribute Report báo trạng thái On/Off mới.

Bước 4 – Cập nhật kết quả về Cloud: Gateway nhận được Attribute Report của đèn, publish bản tin lên kênh `reported` để cập nhật database trạng thái thiết bị, đồng thời gửi bản tin reply mang trạng thái `executed` có `correlation_id` khớp mã lệnh ban đầu. Cloud Backend nhận bản tin này để đánh dấu hoàn tất quá trình điều khiển.

Mọi luồng xử lý trên đã được kiểm chứng hoạt động chính xác thông qua mã nguồn kiểm thử và logs trace ghi nhận tại thư mục `gateway/`.

4.5 Hướng thiết kế triển khai cập nhật OTA

Cơ chế cập nhật firmware từ xa (Over-the-Air - OTA) được phân tích và thiết kế theo đặc tả tài liệu `docs/OTA_CAMPAIGN_CONTRACT.md`. Trạng thái hiện tại của tính năng này là đã hoàn thành thiết kế các hợp đồng truyền dẫn bản tin điều khiển qua MQTT, tuy nhiên mã nguồn triển khai chạy thực tế trên thiết bị vô tuyến được hoãn lại để triển khai ở giai đoạn tiếp theo.

Nguyên tắc thiết kế cốt lõi của OTA là: ****Firmware binary không bao giờ được gửi trực tiếp qua MQTT**** để tránh gây quá tải đường truyền Broker. Thay vào đó, quy trình OTA diễn ra như sau: **Bước 1 – Tạo chiến dịch và phát manifest:** Người quản trị tạo chiến dịch OTA trên Cloud, publish tệp manifest mô tả bản firmware mới (phiên bản, kích thước, mã hash SHA-256)

qua MQTT topic ota/campaigns/{id}/manifest.

Bước 2 – Tải file firmware về Gateway: Gateway nhúng nhận tin nhắn manifest, tiến hành kết nối qua giao thức HTTP để tải tệp firmware binary định dạng .ota từ máy chủ lưu trữ (thiết kế theo đặc tả docs/FIRMWARE_ARTIFACTS.md) và lưu tạm vào bộ nhớ SB_OTA_DIR trên ổ cứng của Host.

Bước 3 – Xác thực tính toàn vẹn: Gateway thực hiện tính toán hàm băm SHA-256 của file đã tải để so khớp với mã hash trong manifest nhằm xác minh tính toàn vẹn của tệp tin.

Bước 4 – Phát lệnh nâng cấp vô tuyến: Sau khi kiểm tra thành công, Gateway sử dụng ZCL OTA Cluster (phía Server) để phát quảng bá lời chào nâng cấp firmware (OTA Query Next Image Response) xuống thiết bị con. Thiết bị con Zigbee (phía Client) sẽ tải từng block dữ liệu vô tuyến từ Gateway qua mạng mesh và tự động flash bộ nhớ để nâng cấp.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này trình bày quá trình xây dựng kế hoạch kiểm thử, thiết lập cấu hình môi trường thử nghiệm thực tế và thực thi các kịch bản kiểm thử nhằm xác minh tính đúng đắn, độ tin cậy của toàn bộ hệ thống IoT Zigbee Smart Building. Các thử nghiệm bao gồm quá trình gia nhập mạng của các node thiết bị vô tuyến, đồng bộ trạng thái từ biên lên ứng dụng di động (Uplink), truyền gửi lệnh điều khiển từ ứng dụng xuống thiết bị chấp hành (Downlink), vận hành kịch bản tự động hóa cục bộ (Local Automation Rules) và kiểm thử khả năng chịu lỗi của hệ thống khi xảy ra sự cố mất kết nối hoặc thiết bị ngoại tuyến. Qua đó, chương sẽ đưa ra đánh giá phân tích chi tiết về hiệu năng, độ trễ và tổng hợp các hạn chế kỹ thuật hiện tại của hệ thống.

5.1 Kế hoạch kiểm thử

Để đảm bảo toàn bộ hệ thống hoạt động ổn định và đáp ứng đầy đủ các yêu cầu chức năng cũng như phi chức năng đã đề ra tại Chương 3, kế hoạch Kiểm thử được xây dựng bao gồm các nhóm thử nghiệm chính sau:

Nhóm kiểm thử gia nhập mạng (Device Join): Xác minh khả năng quét mạng, bắt tay bảo mật và cấp phát địa chỉ mạng động của Coordinator cho ba loại node thiết bị bao gồm Light (Router), Switch (Router) và Occupancy Sensor (Sleepy End Device).

Nhóm kiểm thử truyền nhận và điều khiển (Uplink và Downlink): Kiểm tra luồng đồng bộ trạng thái tự động báo cáo từ thiết bị lên giao diện ứng dụng và luồng gửi lệnh bật/tắt Đèn từ ứng dụng di động xuống biên vật lý thông qua Gateway nhúng.

Nhóm kiểm thử tự động hóa cục bộ (Local Automation): Đánh giá hoạt động của Rules Engine ở biên khi thực thi kịch bản tự động hóa liên thiết bị mà không cần sự can thiệp của Cloud.

Nhóm kiểm thử khả năng chịu lỗi và tin cậy (Reliability): Đánh giá khả năng tự động khôi phục kết nối, xử lý trường hợp thiết bị mất nguồn đột ngột, và hoạt động của cơ chế quét Timeout trên Cloud.

5.2 Môi trường và cấu hình kiểm thử

Quá trình kiểm thử được thực hiện tại phòng thí nghiệm chuyên đề IoT với các thành phần phần cứng, phần mềm và sơ đồ kết nối mạng được cấu hình chi tiết như sau:

5.2.1 Cấu hình phần cứng thử nghiệm

Hệ thống vật lý thử nghiệm bao gồm các thiết bị sau:

Coordinator (NCP): Sử dụng bộ phát triển vô tuyến Silicon Labs Wireless Starter Kit (WSTK) Mainboard kết hợp với Radio Board BRD4162A mang vi điều khiển EFR32MG12 (năng tần +10 dBm, 2.4 GHz).

Z3Light Node (Router): Sử dụng bo mạch tương tự BRD4162A cấu hình firmware

Router bật cụm On/Off Cluster, LED0 dùng để mô phỏng Đèn chấp hành.

Z3Switch Node (Router): Bo mạch BRD4162A cấu hình cụm On/Off Client Cluster, nút nhấn BTN0 dùng để điều khiển.

Occupancy Sensor Node (Sleepy End Device): Bo mạch BRD4162A, nút BTN0 mô phỏng cảm biến hồng ngoại phát hiện hiện diện của con người.

Gateway Host: Máy tính nhúng chạy hệ điều hành Ubuntu Linux kết nối với Coordinator NCP qua cáp micro-USB sử dụng cổng nối tiếp ảo VCOM UART (tốc độ baud 115200, chế độ điều khiển luồng phần cứng CTS/RTS).

Thông tin chi tiết về các địa chỉ vật lý EUI-64 duy nhất và địa chỉ mạng NodeID 16-bit được ghi nhận thực tế trong quá trình cấu hình mạng Zigbee được thể hiện ở Bảng 5.1.

Bảng 5.1: Cấu hình địa chỉ và vai trò của các thiết bị trong mạng thử nghiệm

Tên thiết bị	Vai trò Zigbee	EUI-64 Address	NodeID	Ghi chú
Coordinator	Coordinator / NCP	00:0D:6F:00:1A:BC:D1:02	0x0000	Thiết lập Trust Cent mạng
Z3Light	Router (Active)	00:0D:6F:00:12:34:56:78	0x5A12	Đèn mô phỏng q LED0
Z3Switch	Router (Active)	00:0D:6F:00:98:76:54:32	0xA3F2	Công tắc cơ học n BTN0
Occupancy Sensor	Sleepy End Device	00:0D:6F:00:11:22:33:44	0x2BC4	Cảm biến nút nh BTN0

Quá trình cấu hình phần mềm và môi trường Cloud được chuẩn bị đồng bộ như sau:

Firmware thiết bị: Sử dụng gecko SDK v4.5.0 và EmberZNet stack.

Gateway Service: Biên dịch C thuần chạy native trên Ubuntu, kết nối cục bộ tới Mosquitto Broker chạy độc lập.

Cloud Backend: Dockerized FastAPI deploy trên một máy chủ ảo AWS EC2 (Instance type t3.micro, OS Ubuntu Server).

Cơ sở dữ liệu: PostgreSQL v16 chạy containerized trên AWS EC2.

MQTT Broker: Mosquitto v2.0.18 cấu hình tắt kết nối nặc danh và kích hoạt kiểm soát quyền truy cập ACL.

5.3 Kết quả thực thi các kịch bản kiểm thử

Để quá trình đối chứng kết quả được rõ ràng, các kịch bản kiểm thử được chia tách thành ba nhóm chính tương ứng với các bảng chi tiết bên dưới.

5.3.1 Nhóm 1: Kiểm thử Gia nhập mạng của thiết bị

Nhóm này kiểm tra tính đúng đắn của quá trình quét sóng, bắt tay và cấp phát NodeID động của lớp mạng Zigbee 3.0. Chi tiết các bước và kết quả được trình bày tại Bảng 5.2.

Bảng 5.2: Nhóm kịch bản kiểm thử gia nhập mạng (Device Join)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-01	Gia nhập Z3Light	Đèn Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Đèn để quét mạng.	Bấm BTN1 trên Đèn	Đèn join mạng thành công, LED0 nháy 3 lần, Gateway nhận bản tin thông báo.	Đèn join thành công, gán NodeID 0x5A12. LED0 nháy báo hiệu.	Đạt	Gateway log: Device join
TC-02	Gia nhập Z3Switch	Công tắc Router kết nối Coordinator	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Switch để quét mạng.	Bấm BTN1 trên Switch	Công tắc join mạng thành công, nhận NodeID từ Coordinator.	Công tắc join thành công, gán NodeID 0xA3F2.	Đạt	Gateway log: Device join
TC-03	Gia nhập Cảm biến	Cảm biến Sleepy End Device kết nối	Chạy lệnh cho phép join trên Gateway. Bấm nút BTN1 trên Sensor để quét mạng.	Bấm BTN1 trên Sensor	Sensor join mạng thành công dưới dạng Sleepy End Device.	Sensor join thành công, gán NodeID 0x2BC4, cấu hình sleep duration.	Đạt	Gateway log: Device join

5.3.2 Nhóm 2: Kiểm thử truyền nhận trạng thái và điều khiển

Nhóm này kiểm tra tính đồng bộ dữ liệu hai chiều Uplink và Downlink giữa thiết bị phần cứng, Gateway nhúng, MQTT Broker, FastAPI Backend và CSDL PostgreSQL trên đám mây. Chi tiết tại Bảng 5.3.

Bảng 5.3: Nhóm kịch bản kiểm thử truyền tin và điều khiển (Uplink/Downlink)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-04	Điều khiển Down-link	Gửi lệnh bật/tắt đèn từ App xuống phần cứng	Người dùng nhấn bật/tắt đèn trên giao diện App di động, gửi lệnh qua REST API.	Nhấn "ON" trên App di động	Cloud tạo command record, publish MQTT command. Gateway nhận, gửi ZCL tới Đèn. Đèn LED0 sáng.	Đèn LED0 bật sáng. Gateway nhận được ZCL response, báo cáo trạng thái thành công về Cloud.	Đạt	CSDL lưu trạng thái 1. MQTT topic sb/v1/hust nhận executed.

Bảng 5.3 (tiếp theo): Nhóm kịch bản kiểm thử truyền tin và điều khiển

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-05	Báo cáo Uplink	Đồng bộ trạng thái Đèn tự động báo cáo lên Cloud	Đèn thay đổi trạng thái On/Off cơ học hoặc đến chu kỳ gửi bản tin báo cáo.	Thay đổi trạng thái đèn trực tiếp ở biên	Đèn tự phát bản tin Attribute Report. Gateway nhận, gửi MQTT reported. Cloud cập nhật DB.	Đèn tự động gửi report. DB cập nhật bản ghi mới của thiết bị Đèn.	Đạt	Gateway log: Received r DB state cập nhật.

5.3.3 Nhóm 3: Kiểm thử tự động hóa cục bộ và độ tin cậy

Nhóm này kiểm tra tính thông minh cục bộ ở biên thông qua Rules Engine và khả năng phục hồi của hệ thống khi đối mặt với các kịch bản lỗi vật lý. Chi tiết được thể hiện ở Bảng 5.4.

Bảng 5.4: Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi (Automation & Faults)

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-06	Tự động hóa Switch	Nhấn Switch tự động bật Đèn cục bộ	Bấm nút BTN0 trên Công tắc. Gateway nhận sự kiện và gửi lệnh ZCL điều khiển sang Đèn.	Bấm BTN0 trên Switch	Cổng Gateway bắt được tin của Switch, đổi chiều rule, gửi ZCL Toggle tới Đèn. Đèn toggle LED.	Đèn thay đổi trạng thái On/Off tương ứng.	Đạt	Gateway log: Switch 0x2
TC-07	Tự động hóa Cảm biến	Cảm biến chuyển động tự động bật Đèn	Bấm nút BTN0 trên Sensor để mô phỏng phát hiện người. Chờ 10 giây để hết chuyển động.	Bấm BTN0 trên Sensor	Sensor báo cáo occupancy = 1, Đèn bật. Sau 10 giây, sensor báo cáo occupancy = 0, Đèn tắt.	Đèn bật sáng tức thì khi bấm BTN0. Sau 10 giây đèn tự động tắt (độ trễ 2.8 giây).	Đạt	Gateway log: Sensor 0x2 Sau 10s: occupancy

Bảng 5.4 (tiếp theo): Nhóm kịch bản kiểm thử tự động hóa và xử lý lỗi

No	Tên test	Mục tiêu	Nội dung chi tiết	Input	Kết quả kỳ vọng	Kết quả thực tế	KQ	Bảng chứng
TC-08	Mất mạng khôi phục	Kiểm tra vận hành khi offline và tự động phục hồi	Rút dây mạng của Gateway. Điều khiển cục bộ. Cắm lại dây mạng.	Ngắt và kết nối lại internet tại Gateway	Mất mạng: rule cục bộ vẫn chạy. Có mạng: Gateway tự kết nối lại MQTT và sync trạng thái mới lên Cloud.	Điều khiển cục bộ qua Switch vẫn chạy tốt khi offline. Sau khi cắm lại, Gateway tự reconnect và sync trạng thái sau 4.5 giây.	Đạt	Gateway log: MQTT con và MQTT rec
TC-09	Quét Time-out lệnh	Xử lý khi gửi lệnh tới thiết bị mất nguồn	Ngắt nguồn điện của Đèn Z3Light. Gửi lệnh bật Đèn từ App.	Gửi lệnh "ON" tới Đèn đã mất nguồn	Gateway không thể gửi ZCL vì thiết bị offline. Cloud quét timeout sau 5000 ms, đổi trạng thái lệnh sang "timeout".	Lệnh ở trạng thái "sent" trên Cloud. Sau 5000 ms, Background Worker quét và cập nhật trạng thái lệnh thành "timeout".	Đạt	DB command record cập nhật state timeout. Log của worker: Command

5.4 Đánh giá kết quả kiểm thử

Thông qua việc thực hiện thành công các nhóm kịch bản kiểm thử hệ thống ở trên, các tiêu chí đánh giá về độ chính xác và hiệu năng vận hành đã được phân tích cụ thể:

5.4.1 Mức độ hoàn thành các tiêu chí hệ thống

Bảng 5.5 tổng hợp mức độ hoàn thành và đánh giá kết quả vận hành thực tế đối với từng luồng chức năng quan trọng của hệ thống.

Bảng 5.5: Tổng hợp đánh giá mức độ hoàn thành các tiêu chí hệ thống

Tiêu chí đánh giá	Kết quả	Nhận xét phân tích
Đồng bộ trạng thái Uplink	Đạt 100%	Dữ liệu báo cáo tự động từ thiết bị được Gateway chuẩn hóa JSON và đồng bộ về CSDL Cloud tức thời.
Điều khiển Downlink	Đạt 100%	Các lệnh bật/tắt thiết bị chấp hành từ App di động được thực thi đúng đắn với phản hồi trạng thái hoàn thành rõ ràng.
Tự động hóa cục bộ	Đạt 80%	Điều khiển liên kết tự động cục bộ (Switch-Light) hoạt động tốt, tuy nhiên rule engine ở biên mới chạy các kịch bản cứng tính.
Xử lý lỗi	Đạt 90%	Đã kiểm thử và chạy ổn định cơ chế tự phục hồi kết nối của Gateway và cơ chế quét dọn các lệnh bị treo (Timeout) ở Cloud.
Tính nhất quán giao diện	Đạt 95%	Trạng thái biểu diễn trên thiết bị Đèn vật lý luôn trùng khớp với giao diện hiển thị của người dùng trên ứng dụng di động.

5.4.2 Phân tích độ trễ của hệ thống

Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong Độ trễ trung bình của các luồng xử lý chính được đo đạc thực tế trong quá trình thử nghiệm (tổng hợp từ 50 lần thực thi mỗi kịch bản):

Độ trễ điều khiển Downlink (Cloud-based): Đo từ thời điểm người dùng nhấn nút trên ứng dụng di động cho tới khi Đèn LED0 trên kit thực tế bật sáng. Độ trễ trung bình đạt khoảng 320 ms (trong điều kiện mạng kết nối tốt), tối đa không quá 780 ms ngay cả khi mạng có hiện tượng dao động jitter.

Độ trễ đồng bộ Uplink: Đo từ thời điểm trạng thái Đèn vật lý thay đổi cho đến khi thông tin cập nhật xuất hiện trên màn hình ứng dụng di động. Độ trễ trung bình đạt khoảng 280 ms.

Độ trễ tự động hóa cục bộ ở biên (Local Rule): Đo từ khi bấm nút BTN0 trên Công tắc cho đến khi Đèn LED0 đổi trạng thái, vận hành hoàn toàn qua Gateway cục bộ mà không gửi tin lên Cloud. Độ trễ đạt khoảng 45 ms đến 60 ms, chứng minh ưu điểm vượt trội về tốc độ phản hồi của việc xử lý tính toán tại biên.

5.5 Hạn chế của hệ thống phát hiện qua kiểm thử

Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và Mặc dù hệ thống đã đáp ứng tốt các yêu cầu kiểm thử chức năng cơ bản và đạt được độ trễ vận hành ấn tượng, một số hạn chế kỹ thuật quan trọng đã được ghi nhận:

Quy mô thử nghiệm nhỏ hẹp: Các phép đo kiểm mới chỉ được thực hiện trong môi trường phòng thí nghiệm nhỏ với số lượng node hạn chế (04 node). Chưa có số liệu đánh giá về độ tin cậy truyền tin, tỷ lệ suy hao gói tin (Packet Delivery Ratio - PDR) và độ trễ cụ thể khi mạng mở rộng quy mô lên hàng chục hoặc hàng trăm node trong tòa nhà nhiều tầng có nhiều vật cản kim loại và bê tông.

An toàn bảo mật chưa hoàn thiện: Giao thức MQTT kết nối giữa Gateway nhúng và Cloud Broker hiện tại mới chỉ dừng lại ở mức xác thực tài khoản/mật khẩu tĩnh mà chưa triển khai mã hóa TLS (Transport Layer Security - Port 8883) để bảo vệ toàn vẹn và chống nghe trộm

dữ liệu trên đường truyền internet công cộng. Giao diện ứng dụng di động Flutter cũng cần bổ sung các cơ chế xác thực người dùng nâng cao và phân quyền truy cập.

Hạn chế của phần cứng cảm biến hiện diện: Trong kịch bản thử nghiệm TC-07, việc sử dụng nút nhấn BTN0 cơ học để mô phỏng ngắt của cảm biến chuyển động PIR chỉ phản ánh được logic phần mềm. Trên thực tế, khi tích hợp các cảm biến hiện diện vật lý thực tế, hệ thống cần xử lý thêm các hiện tượng nhiễu tín hiệu nhiễu nhiệt và các trạng thái chuyển đổi liên tục giữa có người/không có người để tránh việc bật/tắt thiết bị đèn liên tục gây hư hại phần cứng.

Điểm lỗi đơn lẻ (Single Point of Failure): Bộ điều phối mạng Coordinator (NCP) kết nối trực tiếp với duy nhất một tiến trình Z3Gateway nhúng. Nếu bo mạch Coordinator bị lỗi nguồn hoặc tiến trình C trên Gateway bị treo đột ngột, toàn bộ mạng cục bộ sẽ bị cô lập, các thiết bị con không thể liên lạc với nhau và người dùng hoàn toàn mất quyền điều khiển từ xa.

Như vậy, toàn bộ quá trình kiểm thử đã cung cấp đầy đủ dữ liệu thực tế để chứng minh tính khả thi của giải pháp kiến trúc hệ thống IoT Zigbee Smart Building đã thiết kế, đồng thời vạch ra những vấn đề kỹ thuật cốt lõi cần giải quyết ở giai đoạn tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết lại toàn bộ các kết quả nghiên cứu, thiết kế và triển khai thực tế đã đạt được trong phạm vi của đồ án tốt nghiệp. Đồng thời, chương cũng trình bày những bài học kinh nghiệm, kiến thức kỹ thuật gặt hái được trong quá trình nghiên cứu và đề xuất các hướng nghiên cứu phát triển tiếp theo nhằm hoàn thiện hệ thống IoT Zigbee Smart Building trong tương lai.

6.1 Kết luận chung

Sau thời gian nghiên cứu và thực hiện, đồ án tốt nghiệp đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, xây dựng thành công một hệ thống IoT Smart Building đồng bộ từ tầng vật lý đến tầng ứng dụng của người dùng. Các kết quả cụ thể đạt được bao gồm:

Thiết kế và triển khai mạng Zigbee 3.0 ổn định: Xây dựng thành công các firmware chuyên biệt trên dòng vi điều khiển EFR32MG12 bao gồm Coordinator (chế độ NCP), Đèn Z3Light (Router), Công tắc Z3Switch (Router) và Cảm biến hiện diện (Sleepy End Device), đồng thời thiết lập thành công Trust Center bảo mật và cơ chế gia nhập mạng động.

Xây dựng Gateway nhúng hiệu năng cao: Triển khai ứng dụng Gateway viết bằng ngôn ngữ C kết nối trực tiếp với thư viện Paho MQTT C để loại bỏ hoàn toàn các lớp trung gian Unix socket/IPC cũ. Gateway đã thực hiện tốt chức năng phân giải UART EZSP/ASH, định tuyến lệnh và ánh xạ Correlation ID giữa các luồng bản tin.

Phát triển Cloud Backend và CSDL đám mây: Xây dựng hệ thống API RESTful và MQTT Client bất đồng bộ sử dụng framework FastAPI kết nối PostgreSQL, triển khai Worker quét dọn Timeout lệnh ở nền và đóng gói toàn bộ hệ thống bằng Docker Compose trên AWS EC2.

Phát triển ứng dụng di động Flutter: Thiết kế giao diện ứng dụng di động trực quan cho phép giám sát trạng thái thiết bị gần thời gian thực, gửi lệnh bật/tắt Đèn và theo dõi lịch sử sự kiện hệ thống.

Mặc dù quy mô thử nghiệm còn nhỏ và một số tính năng như OTA hay an ninh mã hóa TLS chưa được hiện thực hóa đầy đủ trên phần cứng, hệ thống đã chứng minh được tính khả thi về mặt kiến trúc và là tiền đề vững chắc cho các ứng dụng Smart Building thực tiễn.

6.2 Kiến thức và kỹ năng đạt được

Quá trình thực hiện đồ án đã giúp sinh viên tích lũy và nâng cao nhiều kiến thức chuyên môn cũng như kỹ năng thực hành kỹ thuật quan trọng:

Kiến thức lý thuyết: Hiểu rõ cấu trúc phân lớp của giao thức vô tuyến Zigbee 3.0, mô hình cụm chức năng Zigbee Cluster Library (ZCL), cơ chế hoạt động của giao thức truyền thông MQTT và mô hình cơ sở dữ liệu quan hệ PostgreSQL.

Kỹ năng lập trình nhúng: Thành thạo việc cấu hình, lập trình firmware trên IDE Simplicity Studio v5 của Silicon Labs, làm việc với ngăn xếp EmberZNet stack và lập trình hệ thống bằng C trên Linux.

Kỹ năng Cloud và DevOps: Tiếp cận với các công nghệ xây dựng API bất đồng bộ với FastAPI, quản lý và vận hành CSDL qua SQLAlchemy, đóng gói container Docker và quy trình triển khai máy chủ thực tế trên nền tảng điện toán đám mây AWS EC2.

Kỹ năng nghiên cứu và giải quyết vấn đề: Rèn luyện tư duy phân tích thiết kế hệ thống có tính hệ thống, phương pháp kiểm thử thực tế dựa trên logs, khắc phục các lỗi phát sinh thực tế như CRLF dòng lệnh, xung đột cổng dịch vụ hay phân quyền ACL.

6.3 Hướng phát triển tiếp theo

Để hệ thống trở nên hoàn thiện hơn, tiệm cận với các tiêu chuẩn sản phẩm thương mại, các hướng nghiên cứu phát triển tiếp theo được đề xuất bao gồm:

6.3.1 Hoàn thiện phần cứng và mạng thiết bị

Mở rộng loại thiết bị: Nghiên cứu phát triển thêm các loại node thiết bị mới như thiết bị điều chỉnh độ sáng (Dimmer), cảm biến nhiệt độ độ ẩm, công tắc thông minh nhiều kênh và ổ cắm thông minh.

Ổn định hóa Occupancy Sensor: Thay thế nút nhấn BTN0 bằng cảm biến chuyển động vật lý PIR hoặc cảm biến hiện diện radar mmWave thực tế, kết hợp lập trình các thuật toán chống nhiễu (debouncing/filtering) ở firmware để đưa ra trạng thái hiện diện chính xác nhất.

Cấu hình nhóm (Zigbee Group/Scene): Nghiên cứu triển khai tính năng nhóm thiết bị ở lớp mạng để thực hiện các lệnh điều khiển đồng thời (ví dụ: tắt toàn bộ đèn trong phòng) một cách đồng bộ và nhanh chóng.

6.3.2 Nâng cấp tính năng cho Gateway nhúng

Chuyển đổi sang nền tảng máy chủ nhúng: Chuyển tiến trình Z3Gateway từ máy tính xách tay Ubuntu sang chạy thực tế trên các máy tính nhúng như Raspberry Pi hoặc Orange Pi kết hợp với một module NCP EFR32 cắm qua cổng USB.

Hoàn thiện Rules Engine cục bộ: Phát triển bộ thông dịch rule động tại Gateway, cho phép người dùng cấu hình các kịch bản tự động hóa (ví dụ: Đèn tự bật khi Cảm biến báo có người vào ban đêm) ngay tại biên để hệ thống vẫn tự động hóa bình thường khi mất kết nối mạng.

6.3.3 Nâng cấp nền tảng Cloud và Ứng dụng di động

Truyền dữ liệu thời gian thực (Real-time update): Bổ sung cầu nối WebSocket hoặc Server-Sent Events (SSE) giữa Cloud Backend và App di động để thay thế cho cơ chế kéo (poll) dữ liệu REST API hiện tại, giúp cập nhật trạng thái tức thời với băng thông tối thiểu.

Triển khai tính năng OTA đã thiết kế: Lập trình Gecko Bootloader trên thiết bị con và tích hợp logic nạp GBL file vô tuyến tại Gateway để hoàn thiện luồng nâng cấp phần mềm từ xa (OTA Campaign).

Đẩy thông báo (Push Notification): Tích hợp Firebase Cloud Messaging (FCM) để gửi thông báo khẩn cấp tới điện thoại người dùng khi phát hiện chuyển động bất thường hoặc khi có thiết bị ngoại tuyến.

6.3.4 Tăng cường An ninh bảo mật

Mã hóa TLS cho đường truyền MQTT: Thiết lập chứng chỉ bảo mật SSL/TLS trên Mosquitto Broker và cấu hình Gateway/Cloud kết nối qua cổng 8883 để bảo mật toàn bộ dữ liệu trao đổi.

Xác thực người dùng nâng cao: Triển khai cơ chế xác thực OAuth2 kết hợp JWT (JSON Web Token) trên Cloud API để bảo vệ các endpoint điều khiển khỏi truy cập trái phép.

TÀI LIỆU THAM KHẢO

- [1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.
- [2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.
- [3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.
- [4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.
- [5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.
- [6] Gryglecki, M. (HiveMQ). (n.d.). *Speaker Deck: MQTT Broker 100 - Basic*.
- [7] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.
- [8] Circuit Basics. (n.d.). *Basics of UART Communication*.
- [9] Texas Instruments. (2010). *KeyStone Architecture Universal Asynchronous Receiver/-Transmitter (UART) User Guide*. Literature Number: SPRUGP1.
- [10] Silicon Labs. (n.d.). *UG261: EFR32MG12 2.4 GHz 10 dBm Radio Board User's Guide*. (Dành cho bo mạch BRD4162A và các kit liên quan).
- [11] Silicon Labs. (n.d.). *UG489: Silicon Labs Gecko Bootloader User's Guide for GSDK 4.0 and Higher*.
- [12] Silicon Labs. (n.d.). *Gecko Platform Release Notes 4.4.5.0*.
- [13] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.
- [14] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.
- [15] Google. (n.d.). *Firebase Documentation*. <https://firebase.google.com/docs>.
- [16] Google. (n.d.). *Firebase Realtime Database*. <https://firebase.google.com/docs/database>.
- [17] Google. (n.d.). *Flutter Documentation*. <https://docs.flutter.dev>.
- [18] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.